International AI Track | Technical Document

Required material for the International AI Track. Please complete this document in English.

Team name	[TEAM: team name]
Competition track	International AI Track, Topic E: SME Finance Copilot [TEAM: confirm wording]
Submission date	[TEAM: submission date]

TermsLedger is a simulation of an agent owned by a small supplier: lenders, the buyer and the bank feed are simulated, and it makes no claim of bank-covenant compliance. It manages both of the supplier's debt books under limits the owner has signed. **Terms Guard** covers the credit the supplier gives buyers through payment terms, discounts and rebates. **Debt Desk** covers the credit it takes from lenders to survive those terms. The language model only interprets text. Deterministic code decides and executes.

Data notice. Every company, buyer, lender, rate, amount and message in this document is a **synthetic fixture** (`fixtures/`). Every figure quoted comes from `fixtures/golden/golden_values.json` (an independent oracle, `fixtures/golden/compute_golden.py`) or from a report regenerated at the submitted commit: `services/api/evals/latest_report.md` (the full offline evaluation), `services/api/evals/accuracy_summary.md` (the only compiler-accuracy claims we make), `services/api/evals/security/latest_report.md` (the attack suite) and `services/api/evals/e2e/latest/summary.md` (the scripted tasks). The test counts are the ones the repository README states for 12 Oct 2026, and each figure below names its source in a parenthesis. No real customer, bank or transaction data is used. The source is at [TEAM: public repository URL and the commit or tag submitted].

01 Project Overview

- Problem, target users and selected topic

Selected topic. International AI Track, **Topic E: SME Finance Copilot** (financing and cash-flow support for small businesses), under the 2026 theme "AI as a Financial Participant".

The problem. Large buyers now negotiate payment terms through software. Pactum's public material, for example, describes payment-term, discount and rebate agents that operate "within parameters you define". Lenders approve small-business loans by model in seconds (MYbank, WeBank). The supplier in the middle usually has neither: just an owner reading a chat message at night, deciding alone whether to accept 60-day terms and, separately, whether to borrow.

The reframe that drives the product: **a payment-term extension is an interest-free loan from the supplier to the buyer, and the supplier funds it by borrowing.** On the synthetic fixture (a ¥420,000 monthly invoice to the largest buyer):

- moving that buyer from 45 to 60 days going forward ties up **¥210,000.00** of extra receivables, permanently;

- funding that on the cheapest committed source that can carry a permanent need (invoice financing at **10.8417%** all-in, since the credit line's clean-down rules it out) costs **¥1,871.30 per invoice**, or **¥24,326.92** over the 13 invoices dated inside a 365-day window (a monthly service of ¥1,897.29);
- the extension moves ¥0 today, so it passes every spend cap. Wallet-style controls (spend caps, allowlists, signer policies) guard transfers; none of them can see a promise.

Target users. Supplier SMEs with roughly 20–100 staff, 10–40 buyers, one bank credit line and at least one term loan, a bookkeeper and no finance team: light-manufacturing subcontractors, packaging, logistics, MRO and facility-services suppliers. This profile is a composite (spec v2 §3.1) and is labelled **INFERENCE**. Two real SME interviews contradicted one of our assumed pain points (that the owner phones the bank when cash looks low), so we do not claim it. [TEAM: add any validated user interviews, or leave this as stated.]

- Core value and innovation

Two features, one engine.

- **Terms Guard (the credit the SME gives).** It compiles a buyer's free-text offer (Chinese or English, message plus attachments) into typed obligations with source spans. It projects them onto a dated daily cash, debt and forward-concession trajectory, prices every concession at the firm's own marginal cost of funds, and returns ACCEPT, REJECT, COUNTER, NO DEAL, NEEDS_OWNER_APPROVAL or NO_VERDICT. A formula and its inputs back every reason.
- **Debt Desk (the credit the SME takes).** It sizes the gap from the same trajectory, compares the owner's own facilities on all-in cost, time to cash and covenant effect (DSCR, leverage, clean-down), and picks the cheapest admissible plan. It also evaluates capex financing structures and prepares a borrower file for the owner.

Each feature sharpens the other. Concessions are priced at the SME's real marginal cost of debt. A known funding alternative lets the agent walk away (NO DEAL) and still cover the gap.

What the agent actually does. Inside three permission tiers, it counters and rejects offers, accepts the one kind of offer it can prove it has read completely, records commitments, draws and repays committed facilities, and submits invoices for financing. Everything else waits for the owner. Every action is a typed operation in an append-only, hash-chained log.

The agent stays on duty after the decision. The books are current: every accepted commitment changes every later projection, and a simulated clock (`POST /clock/advance`) moves the books forward day by day. Receipts come only from the bank feed, so a payment the buyer promised and did not make is visible. Partial settlement is recorded (a partial payment leaves the residual open). Reconciliation reports each promised early payment as AWAITED, RECEIVED, LAPSED or SETTLED_OTHERWISE. An alert can carry a typed cure that the owner runs with one call (`POST /alerts/{id}/act`). In chat the agent speaks first (`POST /nl/opener`): it says what needs the owner, proposes a cure inside the mandate with its cost, effective annual rate and the day the cash lands, and runs it only when the owner replies with a direct "go ahead", as exactly the operations shown. A cure whose effective annual cost is above the mandate's ceiling is queued for the owner (Tier 2), and runs only when the owner approves it as a second step. When a drawn credit line can be repaid and cash stays above the floor, the agent proposes the repayment too. On the golden books the path is: the deal on Counter A, the agreed early payment of ¥255,013.80 on day 2, the clock to day 34 with ¥162,800.00 still unpaid, the opener (a ¥200,000.00 advance on invoice RI:BUYER-1:2, cost ¥1,782.19, 32.5% a year for the 10 days from landing to the due

day, above the 15% ceiling), "go ahead" (queued as APR-000001, nothing runs), "approve APR-000001" (the advance runs; its cash lands on day 37), the late residual on day 36, the financed invoice paid on day 47 and its advance repaid from the receipt, the proposal to repay the ¥149,986.20 line draw, reconciliation, and a ledger of 14 hash-chained entries (`scripts/demo_run.py`, 3 Oct 2026).

What the owner sees (the pages as of 12 Oct 2026). Every Business page answers one question in its heading, in the server's own sentence, shows one picture where the data supports one, and has at most one raised surface, the action; everything else sits unboxed on hairlines (`apps/web/DESIGN.md`, "Page pattern, A1"). Home says, for example, that the cash reserve becomes unsafe, draws cash against FLOW's plan on one chart, and offers one recommended step with its cost and whether it is within the signed limits (the Home screenshots under `docs/screenshots/ui-overhaul-2026-10-12/`). The browser computes no money and no date words: every amount and every "when" is a display string written by the server in English and Chinese (INVARIANT 13, `docs/INVARIANTS.md`; `app/api/display.py` and `app/api/presentation.py`, handoff 08 section 4). The pages are Home, Deals, Cash and funding, Receivables and Activity, with Approvals, Documents, Limits and the decision record behind them (handoff 09 section 2).

Decisions have a lifecycle, kept in a record (ADR-0011). An independent flow audit (4 P0, 14 P1, 10 P2 findings) was reproduced finding by finding and all 28 now read FIXED; no engine module was edited and a test pins that no engine module imports the record (`docs/UX_REMEDIATION_REPORT.md` sections 1 and 5; `tests/decisions/test_isolation.py`). The record derives each deal's and each cure's state (thirteen states) from the stores the engine already fills, plus three append-only facts that cannot be re-derived: when a verdict was made, which counter was sent, and the economics the owner was shown when an action was queued (migration 0006). A counter is sent bound to the verdict, the counter and the books version the owner saw; a mismatch is refused and creates nothing (`docs/UX_REMEDIATION_REPORT.md` sections 3 and 5, `docs/SYSTEM_WALKTHROUGH.md` section 12).

Contracts the owner confirms (ADR-0012). The owner uploads a signed buyer contract or a lender agreement; the agent reads it with every term beside its quoted clause, labelled "AI-generated, for reference". The owner confirms or rejects it. The decision is an owner-only operation on the hash chain, bound to the hash of the text shown, and it changes no books; a confirmed contract is a record that the Deals page sets beside the live books (`docs/adr/0012-contract-records.md`; README, features table). A later confirmed buyer contract marks the earlier one superseded (handoff 09 section 2). **What is not claimed:** no reading-accuracy figure for contracts. The demo documents were written to match the synthetic books and the reading grammar was written against them; the held-out contracts are not yet written, and the report says so (`services/api/evals/contracts_heldout_report.md`; ADR-0012, Consequences).

Strategy answers behind flags, read-only. Three answers are computed on labelled copies of the books and change nothing: what the owner can offer a named buyer (the Safe Deal Envelope), what cash arriving earlier is worth (the cash price curve) and whom to chase first (collections marginal value). Each is behind a flag that defaults to off (`FEATURE_ENVELOPE`, `FEATURE_PRICE_CURVE`, `FEATURE_COLLECTIONS_WORTH` in `app/config.py`), and the web shows them only with `NEXT_PUBLIC_STRATEGY_MVP=1`; with the flags off the routes answer 403 `FEATURE_DISABLED` (`docs/STRATEGY_MVP_REPORT.md` section 1). Nothing executes from them: no operation, no tier, no mandate change.

Nothing executes outside the signed mandate. Every operation still passes the executor's gates; the contract decision, the decision record and the strategy answers add no way to move money (ADR-0012; `docs/STRATEGY_MVP_REPORT.md` section 1).

Innovation, stated honestly. Nothing here is a new algorithm. Negotiation agents, cash-flow forecasting, loan marketplaces, covenant monitoring and rebate accrual all exist.

- **New application of known mechanisms.** The supplier's side of buyer-side negotiation automation, joined to the borrower's side of SME lending: control what the agent can *promise*, and the borrowing those promises cause.
- **One partial mechanism-level contribution.** A contingent concession is counted as a bounded maximum-exposure liability, and an unapproved facility as zero, on a dated liquidity trajectory that is used as a *hard acceptance boundary* together with the aggregate concession cap and the bank's covenants.
- **Autonomy requires proof (ADR-0005).** The agent may accept a counterparty's offer on its own only when the compiler can show it read every figure and request in the message. The default is the owner.
- **Integration.** Compiler, trajectory engine, funding router, covenant model, signed mandate, commitment lifecycle and an injection-resistant typed interface, all in one deterministic pipeline.

Prior art we checked (vendor claims unless stated; absence from our search is a search result, not a fact):

System	What it does	Difference from TermsLedger
Pactum	Buyer-side procurement agents negotiating payment days, discounts and rebates within buyer-set parameters	The other side of the table
MYbank 百灵 2.0	An agent that tailors financing plans for about 42 million micro merchants, using MYbank's own credit and bill products only	Lender-owned, single-provider; no link to the trade terms that created the need
MYbank AI 票据管家	Bill-discount price and timing advice	A lender tool for one funding source
Xero JAX Cash Flow Actions (beta, Jul–Aug 2026)	Spots cash-flow gaps and suggests plans (protect payroll, delay bills, bridge gaps)	Ledger-side forecasting; no offer intake, no mandate, no covenant-bounded acceptance
Cerebro Capital	US mid-market debt capacity, covenant compliance tracking and a lender marketplace	Mid-market, lender-network model; no offer intake
Capitalise	UK lender marketplace with credit data	Multi-provider comparison, but no forecast or acceptance boundary
Codat Working Capital Insights	Flags working-capital and refinancing needs to bank relationship managers	Tells the bank, not the SME
Lendio; Cardo AI; Fence	Loan matching; lender-side covenant monitoring; covenant-checked drawdowns for lenders	Start after the owner wants a loan, or serve the lender

Our positioning line is therefore narrow: **none we found is owned by the supplier and bounded by its own mandate across both debt books.** [TEAM: re-check the Chinese-language prior art (supply-chain finance platforms, ERP vendors) before using this line on stage; spec v2 §9 marks it UNVERIFIED.]

Claims we make (spec v2 §13):

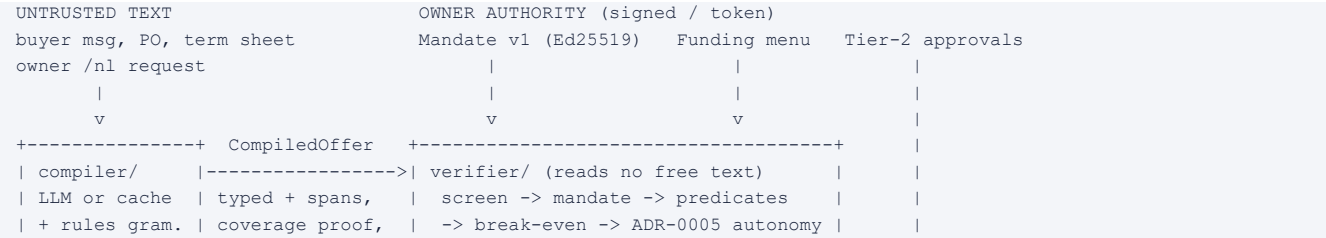
1. No model output alone can create a commitment, borrow or move money in this system. Every operation is built by code from a verifier verdict and gated by the executor.
2. After ADR-0005, no misread message was accepted without the owner, across our three held-out sets and both compiler paths (section 03). Free-text understanding itself is still imperfect, and we say by how much.
3. Every accepted, countered and refused offer carries a typed record with the failing predicate and its source spans.
4. The verifier never reads free text.
5. Contingent concessions count at maximum exposure, and uncommitted facilities at zero, by default.
6. Every funding choice shows its all-in rate, time to cash and covenant effect.
7. A document the agent read never changes the books: the owner's confirmation is a hash-bound record on the chain, and the deal summary only reads it (ADR-0012).
8. The strategy answers are read-only and flagged off by default; they compute on labelled copies and execute nothing (`docs/STRATEGY_MVP_REPORT.md` section 1).

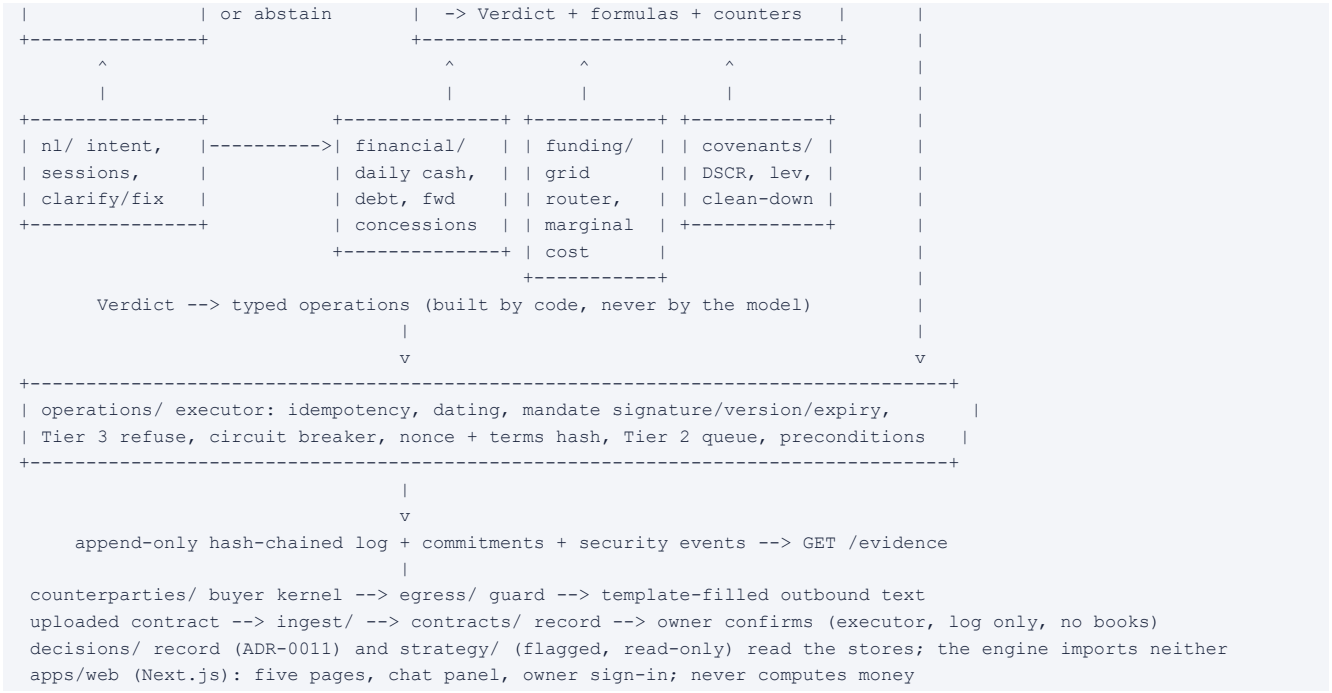
We do not claim "first", "best loan", "proof", "atomic", "financial advice", "bank-grade", "safe by construction", "cannot be fooled" or compliance with any bank's covenants, and we make no reading-accuracy claim for contracts (the held-out set is pending). The checks screen the obligations that were entered, not the whole business; a plan is the cheapest within the bounded search; the bullet-loan DSCR is a reserve-coverage reading; the hash chain is internally checkable only. TermsLedger is decision support under the owner's own limits; the owner decides and signs, and the agent never applies for, signs or guarantees a loan.

02 System Architecture

- Architecture diagram and component responsibilities

TermsLedger is a **modular monolith** (ADR-0001): one FastAPI service (`services/api`), one Next.js web app (`apps/web`) and one database. Shared contracts live in `app/domain/`. Modules talk only through those contracts, and import cycles are forbidden. The diagram's source is `docs/deliverables/architecture.mmd`.





Four roles. The pipeline has four roles. Each is a module with its own boundary, not a free-running LLM agent:

Role	Modules	May do	May not do
Compiler (the only model-backed role, untrusted)	`compiler/`	Turn text into typed obligations with exact spans, flag authority claims, data requests and unknown lenders, and say whether it can prove coverage	See the state, mandate, funding menu or reservations; emit an operation, a tier or a reply
Risk / Verifier	`financial/`, `covenants/`, `verifier/`	Project the trajectory, test the mandate and covenants, return a verdict with formulas	Read free text
Negotiator	`verifier/counters.py`, `counterparties/`, `egress/`	Price counters from the marginal cost of funds, run bounded rounds with the buyer kernel, fill outbound templates	Reveal the floor, caps, menu or reservation values
Treasurer	`funding/`, `operations/`	Find the cheapest admissible funding plan, and execute typed operations through the tier gates	Use an uncommitted facility for a hard constraint; execute Tier 3

Component responsibilities (all under `services/api/app/` unless noted):

Component	Responsibility	Trust
`compiler/`	Turns buyer offers and lender term sheets or agreements into typed obligations or facility terms, and a signed buyer contract into a reading of 18 fields, each with its clause. Every obligation	Untrusted output; schema-validated

Component	Responsibility	Trust
	and term carries a span that must match the document exactly; otherwise the field is abstained or left blank. A zh + en rules grammar flags authority claims, data requests and unknown lenders. The coverage proof (`coverage.py`) sets `autonomy_eligible`.	
`financial/`	The daily cash recursion, debt ledger and forward-concession valuation, in integer fen, in conservative and expected modes	Trusted, deterministic
`covenants/`	DSCR, leverage and clean-down per facility definition, evaluated monthly on the projected state	Trusted, deterministic
`funding/`	The Debt Desk router: grid search for the cheapest admissible mix; marginal cost of funds; break-even early-pay discount; the terms extension priced as a loan	Trusted, deterministic
`mandate/`	Canonical serialisation, Ed25519 signing and verification with a pinned owner key, validity (expiry, revocation, version), tier computation, mandate predicates	Trusted; signed by the owner
`verifier/`	The verdict algorithm, reason codes, formulas and counters priced from the marginal cost of funds. It imports no text-bearing field (enforced by an AST test).	Trusted, deterministic
`operations/`	The executor: the **only** writer of commitments and operations. It computes the tier itself and runs every draw, advance and repayment through admission (`funding/admission.py`: the planner's engine and predicates on today's books), whoever asks.	Trusted
`commitments/`	Commitment lifecycle (PROPOSED → ACCEPTED → ACTIVE → FULFILLED, or EXPIRED / REVOKED), terms hash, nonce, expiry	Trusted
`books/`, `clock/`	`books.current_books` is the one set of current books: the state today plus the live obligations of accepted commitments. Every projection in every module runs on it (ADR-0006). Accepted terms and prices are written onto the invoices once, with a scope fixed at acceptance (ADR-0009), and every draw or advance is a dated funding lot whose cash lands after the lead time (ADR-0010). `Runtime.advance_to` is the only way time moves: it posts the day's scheduled flows, lands funding, posts loan instalments, the existing loan's payments (which leave cash) and maturities, expires lapsed commitments and logs one `ADVANCE_CLOCK` entry on the chain. It never records a receipt and never repays a credit line.	Trusted, deterministic
`persistence/`	SQLAlchemy tables, Alembic migrations (`0001_initial`, `0002_enable_rls`, `0003_append_only_triggers`, `0004_approval_decided_day`, `0005_pilot_journal`, `0006_decision_facts`, `0007_contract_records`) and `LedgerStore` (SQL and in-memory). The operations log is append-only and hash-chained.	Trusted
`counterparties/`	A deterministic buyer kernel with a hidden reservation and a template verbaliser	Separate fixtures; can refuse
`egress/`	Scans every outbound text for protected values and planted canaries, and blocks the whole message on a hit	Trusted, last step
`nl/`	`/nl` harness: natural-language request → intent and typed slots	Trusted orchestration; never skips

Component	Responsibility	Trust
	→ verifier → typed operations; multi-turn sessions with clarification, correction and a nearest-capability fallback. A counterparty's response carries the reply and the session id only, and the counterparty is whoever its `X-Counterparty-Key` names (no key: a proposal only). A session is bound to the kind of caller that opened it (binding it to the individual credential is open, SEC-21). Free text executes only as a closed affirmative command (`consent.py`); plans are proposed first and "go ahead" runs the exact operations shown.	the verifier
`security/`	Attack catalogue runner, security-event helpers, refusal templates	Test and audit
`capex/`, `borrower_file/`, `alerts/`, `whatif/`	Capex NPV and financing structures; a lender-ready borrower file built from verified state (HTML/DOCX, neutral styling; a read-only owner action with its provenance and an evidence hash, never queued for approval); cash alerts, each with a typed cure, and receipt reconciliation including promised early payments; a read-only what-if sandbox. All of them read current books.	Trusted, deterministic
`ingest/`, `contracts/`, `deals/`	Contract ingestion: text from a Word file or a PDF with a text layer (scans refused); a contract record per uploaded document (the reading, its flags, the hash of the text, PROPOSED until the owner confirms or rejects it); the deal summary (on Deals once a deal is agreed), which sets a confirmed contract beside the live books. A document never writes the books	Trusted, deterministic; the reading inside is untrusted model output
`decisions/`	The durable decision record (ADR-0011): three strict fact models (EVALUATED, SENT, QUOTED), deterministic decision ids, the pure `derive()` and `waiting_for()`. Pure, no I/O, no money; imports only `domain` and `persistence.records`; no engine module imports it (`tests/decisions/test_isolation.py`)	Trusted, deterministic
`strategy/`	The three flagged, read-only answers (envelope, cash price curve, collections worth): sweeps, deltas and rankings on labelled copies of the books through the kernel's own `evaluate`, `plan`, `project` and `alerts`; one answer cache. It writes nothing	Trusted, deterministic; flags default off
`pilot/`	The shadow pilot's journal (migration 0005); `tests/pilot/test_isolation.py` pins that the engine does not import it	Trusted
`api/`	FastAPI routers, dependency wiring and authentication (owner token, agent key, bank-feed key). Since map #65 it also writes every sentence the pages show, in both languages (`presentation.py`, `engine_zh.py`, `causality.py`, `workspace.py`, `quotes.py`, `receipt_facts.py`), and holds the routes of the decision record, the chase handoff and the Demo's one synthetic bank receipt	Trust boundary
`apps/web`	Next.js. The Business pages: Home (the answer, one cash picture, the plan bar, the owner's deals), Deals (the verdict as the heading, each ask with its cost, three paths on one chart with a day cursor, the reply in the bar, and once a deal is agreed the confirmed contract beside the books), Cash and funding, Receivables, Activity, and Approvals, Documents (upload, the review screen with every clause beside its term, confirm or reject), Limits, the decision record, What-if and Equipment behind them. The Demo mode keeps the engine's own screens	Displays server-formatted values only

Component	Responsibility	Trust
	(Mandate, Inbox, Trajectory, Funding, Verdict, Ledger, Security, Evidence). A slide-over chat panel on every page, an Owner sign-in dialog that keeps the token in `sessionStorage` only, English and Chinese, and the six-scene guided Demo. Plain SVG pictures drawn from the server's series; every figure and sentence is the server's	

One offer, end to end.

1. A buyer message arrives. The compiler proposes obligations with spans, or abstains. A conflict between the body and an attachment, or a stated discount sum that is not rate × face, gives `NO_VERDICT` with `CONFLICTING_SOURCES`. The compiler then runs the coverage proof.
2. The verifier's security screen logs and neutralises unknown senders, funding claims, unevidenced "we agreed last month" concessions and data requests. None of them can book anything.
3. The engine projects the conservative trajectory under the offer's bookable obligations.
4. The router prices any gap from committed facilities and emits the marginal cost of funds; forward concessions are valued at that rate.
5. The verifier tests every day and month against the mandate and covenants, applies the break-even rule and the ADR-0005 autonomy rule, and returns a verdict with formulas. On REJECT or COUNTER it attaches counters priced from the marginal cost.
6. The buyer kernel answers within bounded rounds; NO DEAL is a first-class outcome.
7. Code, not the model, turns the verdict into typed operations. The executor gates them, runs Tier 1, queues Tier 2 for the owner, refuses Tier 3, and appends every outcome of an admitted operation to the hash-chained log (requests refused before the executor are recorded differently; Security Self-Assessment, 1.9).

Binding design decisions. Money is integer fen and rates are integer basis points, with exact fractions in between and one half-up rounding per posting (ADR-0002). Funding is requested on the decision day and lands after its lead time, so hard constraints never rely on future availability (ADR-0003). The evidence export's hash covers `prev_hash + payload_json + result_json` only, so a cache-mode replay reproduces the same head hash (ADR-0004). Autonomy requires positive proof (ADR-0005). A decision's lifecycle is derived from stored reality, never from a clicked button, and three facts that cannot be re-derived are kept in an append-only table that is not on the hash chain (ADR-0011). A confirmed contract is a record the deal summary reads, never the books (ADR-0012). **The engine stability set** (`app/financial`, `verifier`, `funding`, `alerts`, `operations`, `clock`, `books`, `mandate` and `domain/models.py`) changes only when a reproduction proves cause or the owner grants an explicit exception; the one exception so far is ADR-0012's additive types and executor operations (handoff 08 section 4). The UI never computes money (INVARIANT 13).

- Models, tools, data sources and external services

Models. The model is used only to (a) compile offers, term sheets and contracts into a strict JSON schema and (b) optionally classify an `/nl` request into one intent from a closed list when the rules parser finds nothing. It never decides, prices, negotiates or executes.

Backend	Model	When
`cache` (default)	Recorded outputs keyed by sha256(documents + prompt version) in `fixtures/compiler_cache/`	Tests, CI, the demo, the scenario runner and the offline evaluation. No network. A message that is not in the cache goes to the rules grammar.
`claude`	`claude-sonnet-5-5` by default, `claude-opus-5-5` for hard compilations (Anthropic SDK)	`LLM_MODE=live`, opt-in
`deepseek`	`deepseek-flash` (OpenAI-compatible SDK)	`LLM_MODE=live`; a China-reachable backend. The only live accuracy run so far used it (section 03).
rules grammar	Deterministic regex and lexicon, Chinese and English	Always runs beside the model; the fallback when a backend errors, times out or returns invalid JSON; the path for every uncached message offline

The model sees only the documents and the output schema. Mandate values, facility limits, reservation values and other buyers' terms never enter a prompt; a test checks the compiler package's imports to keep it that way. Only a document hash and timing are logged per call, never the text.

What is redacted before the model. In live mode the product's model calls to Claude or DeepSeek (offers, term sheets, contract chunks and the `/nl` classifier) first pass through `app/compiler/redaction.py`, which replaces the common formats of identifiers FLOW never reads with stable placeholders such as ⟦ACCOUNT_1⟧: bank account and card numbers, phone numbers, email addresses, the unified social credit code, mainland ID and passport numbers, and addresses that follow a label (住所, 地址, Address:). Amounts, dates, rates, contract and invoice numbers and the clauses are sent, because FLOW reads them. The redaction keeps an offset map: the model quotes the redacted text, and each quote is mapped back to the original document's own words before grounding, so every stored span refers to the original; recorded readings, the rules grammar, the coverage proof and the cross-check use the original text and never leave the machine. `LLM_REDACT` (on by default) controls it, and only counts per kind are logged (`model.redaction`). Redaction is a privacy control, not the security boundary. Every provider backend redacts its own input by default, however it is built (PRIV-1 fix, merging); a labelled address ends at its sentence and a labelled short number is not taken for an identifier (PRIV-2, PRIV-3). It is pattern-based and cannot be complete: person and company names, file names and chat ids still reach the provider. Details and limits: Security Self-Assessment, section 03. Tests: `tests/compiler/test_redaction.py`.

Tools (the agent's typed interface). Every tool is an HTTP route with a strict Pydantic schema (`extra="forbid"`):

Tool	Route	
Natural-language harness	`POST /nl` (a buyer sends `X-Counterparty-Key`); the agent's opener `POST /nl/opener` (owner)	

Tool	Route	
Clock, alerts and reconciliation	`POST /clock/advance` (owner token or bank-feed key); `GET /alerts`; `POST /alerts/{id}/act` (runs the alert's cure as the agent; the executor decides the tier); `GET /reconciliation`	
Offer intake and evaluation	`POST /offers`, `POST /offers/{id}/evaluate`, `POST /negotiate/{offer_id}`	
Debt Desk	`POST /router/plan`, `POST /facilities/compile` (owner), `POST /capex/evaluate`, `POST /borrower-file`	
Execution	`POST /operations`, `GET /approvals/queue`, `POST /approvals/{id}`, `POST /circuit/{id}/reset`, `POST /commitments/{id}/revoke` (owner)	
Read views and evidence (owner token)	`GET /state`, `/mandate`, `/trajectory`, `/ledger`, `/audit`, `/evidence`; `POST /demo/reset`	
The pages' reads (owner token)	`GET /home` (its receivables block carries the page's headline), `GET /nav/counts`, `GET /receivables/{id}/chase`, `GET /offers/{id}`, `GET /offers/{id}/picture` (the offer as asked and each reply, as cash lines), `POST /router/plan` (rows carry the cash line if that plan ran); every one answers `?lang=en`	zh `with the same figures` (docs/ARCHITECTURE.md`, API section; handoff 09 section 2)
Decision record (owner token)	`GET /decisions`, `GET /decisions/{id}` (ADR-0011)	
Strategy reads (flags default off)	`GET /buyers/{id}/envelope`, `GET /cash/price-curve`, `GET /receivables/worth` (403 `FEATURE_DISABLED` when off; `docs/STRATEGY_MVP_REPORT.md` section 1)	
Contracts (owner token)	`POST /documents` (a demo document, an upload or pasted text), `GET /documents`, `GET /documents/{id}`, `POST /documents/{id}/confirm`, `POST /documents/{id}/reject`, `GET /deals`, `GET /deals/{buyer}`, `GET /fixtures/contracts`	

The typed operation kinds (`app/domain/models.py`) are:

- ACCEPT_OFFER, COUNTER_OFFER, REJECT_OFFER and RECORD_COMMITMENT;
- DRAW_FACILITY, REPAY_FACILITY, SUBMIT_INVOICE_FOR_FINANCING and RECEIVE_PAYMENT (and, written by the clock as SYSTEM, one ADVANCE_CLOCK entry per move of time);
- APPLY_FOR_FACILITY, ADD_FACILITY and REQUEST_OWNER_APPROVAL (preparing the borrower file is no longer an operation: it is a read-only owner action);

- the Tier-3 kinds SIGN_PERSONAL_GUARANTEE, PLEDGE_COLLATERAL, ISSUE_EQUITY and CHANGE_MANDATE, which exist only to be refused;
- CONFIRM_DOCUMENT and REJECT_DOCUMENT: the owner's decision on the agent's reading of a contract or lender agreement, bound to the hash of the text, logged on the chain, with no effect on the books.

No operation has a payee or account field. A request for a transfer to a third party (`TRANSFER_FUNDS`) is rejected as an unknown kind.

Data sources (all synthetic).

- `fixtures/scenarios/golden_sme.json`: the supplier's state. Cash ¥400,000; payroll ¥340,000 on day 12; supplier payment ¥120,000 on day 20; the existing equipment loan's ¥45,000 payment on day 30 and every 30 days after (it leaves cash); Receivable A ¥420,000 due on day 32; the funding menu below.
- `fixtures/scenarios/golden_sme.mandate.signed.json`: mandate v1, signed with a **published demo key** labelled DEMO ONLY. Floor ¥300,000; aggregate forward-concession cap ¥5,000; DSCR at least 1.25×; leverage at most 2.5×; Tier-2 threshold ¥100,000 per commitment.
- `fixtures/offers/offers.yaml`: the 12 worked examples of spec v2 §8, a reworded EX01, and the FastCapital security message.
- `fixtures/heldout/`, `fixtures/heldout2/`, `fixtures/heldout3/`: three held-out evaluation sets of buyer messages and owner utterances (section 03).
- `fixtures/counterparties/`: buyer profiles with hidden reservations.
- `fixtures/golden/`: the independent oracle and its golden values.
- `fixtures/contracts/`: three synthetic demo documents written to match the golden books (the supply agreement FHR-2026-07 with Foshan Home Retail, the credit line agreement and the invoice financing agreement), each as Markdown, Word and PDF. `fixtures/contracts_heldout/` holds the protocol and the empty label file of the held-out contract set, which is pending.
- `fixtures/scenarios/public_sme.json` and `fixtures/strategy/`: a second set of books built from a public filing, with terms calibrated from surveys because the filing states none, and the strategy layer's spikes and recorded oracle (`docs/STRATEGY_MVP_REPORT.md` sections 1 and 8).
- `services/api/evals/security/attacks.yaml`: 126 attacks through the API; the 133 chat attacks are run through `/nl` by the same runner (`app.security.attacks`).
- `services/api/evals/e2e/scenarios.yaml`: 49 scripted `/nl` tasks.
- `fixtures/scenarios/golden_sme.events.json`: dated events the clock applies. On day 25 the scenario adds the month-2 payroll of ¥200,000 (due day 42); it does not exist on day 0, so day-0 golden values are unchanged.

The synthetic funding menu (owner-approved; only the owner can add to it):

Facility	Status	Price	Lead time
Bank credit line, ¥150,000 limit, 30-day clean-down	COMMITTED	6.4% simple	same day

Facility	Status	Price	Lead time
Invoice financing, onboarded platform, ¥350,000 limit	COMMITTED	7.8% + 0.25% per 30-day period, 10.8417% all-in	3 days
Buyer early-payment offer	COMMITTED	1.2% discount	2 days
Bank acceptance bill discounting (银票)	COMMITTED (no bill held in the base fixture)	0.58% a year	1 day
Government-guaranteed working-capital loan	NOT_APPLIED	5.0% all-in	21 days
Equipment term loan, 36 months	NOT_APPLIED	5.6%	14 days
Equipment lease, 48 months	NOT_APPLIED	7.2%	10 days
Equity	REFERENCE_ONLY (Tier 3)	n/a	n/a

External services. No bank, lender, payment or blockchain API is used; lenders and the buyer are simulated. In live mode only, the Anthropic API or the DeepSeek API receives the inbound documents and the extraction schema. The deployment database is PostgreSQL: the Compose stack's own container, or Supabase (Singapore pooler) through `DATABASE_URL` / `SUPABASE_DB_URL`, with row-level security enabled on every table (migration `0002_enable_rls`, no policies, so Supabase's anon and authenticated roles read nothing) and no Supabase Auth or Storage.

Libraries. Python 3.12, FastAPI, Pydantic v2 (strict), SQLAlchemy 2 + Alembic, PyNaCl (Ed25519), structlog, python-docx, pypdf, openpyxl; Next.js 16, React 19, Tailwind 4; Plus Jakarta Sans Variable bundled with the app, so no font host is called (handoff 09 section 2); Recharts only for the Demo and What-if charts, the cash pictures being plain SVG; pytest, hypothesis and Playwright for tests.

03 Agent Design

- Intent understanding, planning, tool use and state management

Intent understanding (`nl/`).

- A deterministic rules parser (zh + en lexicon) maps each request to one of 21 intents: EVALUATE_OFFER, NEGOTIATE, ACCELERATE_RECEIPT, FUND_GAP, REPAY_FACILITY, SHOW_STATUS, SHOW_ALERTS, COMPARE_FINANCING, RECONCILE, SHOW_RATIOS, CAPEX, BORROWER_FILE, EXPLAIN, WHAT_IF, RECORD_FACT, CHANGES, APPROVE, REJECT, COUNTERPARTY_MESSAGE, OUT_OF_SCOPE and AMBIGUOUS. RECORD_FACT is read-only: when the owner states a new bill, invoice, delay or figure, chat says plainly that it cannot record it and that the forecast is unchanged. It fills typed slots; every id is checked against the registry. Chat therefore reaches the built modules: the financing comparison, alerts, reconciliation, covenant ratios, capex and the borrower file. COMPARE_FINANCING, RECONCILE, SHOW_RATIOS, CAPEX and BORROWER_FILE need

the owner token; a caller with only the agent key is told the owner must sign in. A request that matches no intent gets the nearest capability and what the agent can do, not a flat refusal.

- **Consent grammar.** A financial instruction executes only when the whole message is one closed affirmative command with exactly one RMB amount; negation, a question, a condition, a future time, a foreign currency, an unreadable amount, a rider and quoted or pasted text execute nothing (English and Chinese). An approval id is decided only by an exact decision sentence; a pending plan is confirmed only by a short affirmative. Quoted or pasted text never becomes an instruction or an approval id, and a chat session is bound to the kind of caller that opened it (owner, agent or counterparty).
- **Propose, then confirm.** A plan is stored with the exact typed operations shown, the quoted cost and the books it was built on. "Go ahead" runs those operations, or shows a new proposal when the books moved to a dearer or different plan; a new day or a changed mandate voids it.
- Only when the rules find nothing is an optional LLM classifier consulted (cache mode by default). It may only *choose* an intent from a closed list and fill a strict slot schema. APPROVE, REJECT and COUNTERPARTY_MESSAGE are not in its vocabulary, and a canary in its output discards the answer.
- The actor comes from authentication, never from text. A request from a COUNTERPARTY is never parsed for intents: it is compiled and verified as an inbound message, and it can never execute anything. The counterparty is whoever its channel key names; a message with no key is a proposal only.

Planning. A plan is a list of typed steps built by code from a verdict or a router plan:

- *Goal to plan* (worked example 12): "Get paid early on the Foshan invoice without giving anything away next quarter" becomes an early-pay counter at 0.85% with zero forward concession, and a fallback to invoice financing if the buyer refuses.
- *Multi-step funding*: draw the line, finance an invoice, repay on receipt.
- *Clarification*: "fund it" with two open gaps asks which one; an offer with no sender asks which buyer.
- *Correction*: "make it 0.8%" amends the pending plan; "1%" is refused because it is above the 0.88% break-even; "switch to invoice financing" swaps the source.
- *Out of scope*: the nearest capability and the list of what the agent can do.
- *Proactive re-plan.* After the clock or the bank feed moves the books, the opener names what needs the owner (a late receipt, a reserve breach ahead, a drawn line that can be repaid), proposes a cure and its cost, and waits. The alert's cure is found on current books by a greedy search and priced as the executor will charge it. It runs only on the owner's click or a direct "go ahead", it is submitted as the AGENT, and the executor decides the tier: above the rate ceiling it is queued and a second step (approve) is needed. No cure is offered when the mandate forbids new borrowing. The repay proposal is never made while a reserve breach is open.

Tool use. Handlers call module functions (compile, evaluate, plan, negotiate), then send typed operations to the executor. Every operation carries an idempotency key derived from the session, turn and step. No handler can skip the verifier or build an operation from model output (Invariant 1).

State management.

- *Session state* (`nl/sessions.py`) holds only typed references: the last offer and verdict ids, a pending plan awaiting confirmation or correction, a pending clarification, and the operations each `request_id` produced. A retried request is replayed, not re-planned. Sessions expire after 30 minutes.
- *Financial state* is a frozen `FinancialState` plus the operations log. Contracts are immutable; a state change builds a new model.
- *Time* is an integer simulated day. `Runtime.advance_to` moves it: it posts the day's scheduled flows, turns recurring invoices into receivables, expires lapsed commitments and logs `ADVANCE_CLOCK`. Days stay absolute and the books close every day before `as_of_day`. The executor refuses backdated or future-dated operations.
- *Current books* (ADR-0006). The store holds the position today; commitments hold the agreed changes not yet realised; `books.current_books` joins them, and every projection runs on that. An early payment the buyer did not make by its day drops out of the books and is reported as LAPSED.
- *Settlement.* `Receivable.settled_fen` records partial payment against the contractual face. A receipt equal to the agreed early-payment net, on or before the promised day, settles the covered face and fulfils the commitment (a later receipt is a plain receipt); any other receipt settles what it pays and leaves the residual open on its contractual day. A receipt on a financed invoice pays the lender first, from the receipt only: the advance, then its interest and fee; owner cash never falls on a receipt, and an unpaid remainder of charges stays as a fixed amount the owner or agent repays. An overdue financed invoice, or one fully settled with its advance still open, repays its advance in the projection (recourse).
- *Funding lots* (ADR-0010). A draw or advance takes facility room and creates a dated lot at once; its cash lands after the facility's lead time (net of the upfront fee) and interest runs from the landing day. A funding-in-transit alert names the landing day.
- *Re-verification at commitment.* A verdict is not standing authority. At accept and at approval the executor checks, on the state and mandate of that moment: the named receivables are open with enough face, no other live commitment holds the same invoice, the concession caps still hold, and, if the state version has changed, that the obligations still verify on current books. Otherwise the operation is refused (`STALE_VERDICT` or `CONFLICTING_COMMITMENT`).
- *Commitments* follow a lifecycle state machine with a nonce, an expiry and a terms hash.
- *Decision record* (ADR-0011). The state of a deal or a cure is read from the canonical stores and three stored facts, with deterministic decision ids (`deal:{offer_id}`, `alert:{id}:v{state_version}`, `approval:{id}`); a failed fact write never fails the route, and the read says "not recorded" instead of recomputing history (`docs/UX_REMEDIATION_REPORT.md` section 5). The bound send refuses a changed counter with 409 `COUNTER_CHANGED` and a repeat with 409 `ALREADY_SENT` (`docs/SYSTEM_WALKTHROUGH.md` section 12).
- *Contract records* (ADR-0012). A `ContractRecord` holds the text, the reading with each term's quoted clause, the flags, the sha256 of the text and the owner's decision, in its own table (migration 0007), cleared with the ledger and not part of the state hash. A repeated decision answers with the first one and appends no log row; a different decision is 409 `ALREADY_DECIDED`, a stale hash 409 `TERMS_CHANGED`; extraction is capped (20 MB unzipped, 60 PDF pages, 40 reading chunks) and runs outside the runtime lock (`docs/SYSTEM_WALKTHROUGH.md` section 14).

- Core algorithms and evaluation approach

1. Trajectory recursion (`financial/`). For each day *t* of the horizon:

```
closing(t) = opening(t) + Σ inflows(t) − Σ scheduled outflows(t) − Σ interest, fees and repayments(t)
```

- Within a day, inflows come first (receipts, early receipts, draws, advances), then scheduled outflows, then interest, fees and repayments. `closing(t)` is tested against the floor.
- **Conservative mode decides:** unconfirmed inflows are ignored, only COMMITTED facilities can be drawn (PENDING and NOT_APPLIED count as zero), and contingent concessions count at their maximum. Expected mode is shown beside it and never used for a hard constraint.
- Forward concessions go to a separate forward-concession ledger, valued over the concession horizon at the marginal cost of funds.
- Month *m* covers days 30(m−1)+1 to 30m and carries EBITDA, debt service and total debt for the covenants. The horizon always runs past the last repayment.
- Everything is integer fen. Interest is Actual/365 simple; invoice financing bills whole 30-day periods; each posting is rounded once, half-up (ADR-0002).

Golden check. With the full EX01 package accepted and no new funding, cash is **¥354,960** after day 20 and **¥209,960** on day 47: a **¥90,040** shortfall against the ¥300,000 floor (the existing loan's ¥45,000 payments leave cash on days 30 and 60). The committed marginal source cures it for **¥2,406.67**.

2. Funding router (`funding/search.py`). "Cheapest" means cheapest within the bounded search below (at most three actions on a ¥10,000 grid), checked against a brute force; it is not a proof of a global optimum. Each candidate is a list of funding actions run through the same engine. Its cost is whatever that trajectory posts (interest, fees and discounts, net of what the offer alone posts), so the router never computes cash itself.

- **Admissible** only if closing cash stays at or above the floor every day, no covenant is violated, and new borrowing is allowed. A leg whose effective annual cost (one function, `funding/rates.py`, shared with the executor) is above the mandate's `max_apr_bp` does not make a plan inadmissible: the plan is flagged `needs_owner_approval`, and a plan that needs no one ranks before one that does. A plan that uses an invoice not yet issued, or that is admissible only as a whole, is flagged `executable_now` false ("cannot run today"). The router orders a plan's legs so that every prefix is admissible, judging each leg on the books the legs before it leave with the one rule admission applies (`funding/legs.py`).
- **Timing.** Every source is requested on day 0 and lands after its lead time (ADR-0003). The search is over amounts, not dates, on a **¥10,000 grid**: single-source fills, then pairs (a grid over the first source, the second filling the residual), then triples.
- **Repayment on any receipt day.** A bullet line may be repaid on any receipt day after the gap opens on which cash stays at or above the floor, not only on the day the unfunded baseline recovers (ADR-0003 addendum; evaluation finding EV-06).
- **¥1,000 minimum action** (`MIN_ACTION_FEN`). No draw, advance, early payment or bill discount is smaller; a smaller residual is rounded up, and a source with less than ¥1,000 available is not used. This removed plans such as a 1-fen early payment for a 1-fen shortfall (EV-11).
- **Cash-path fast check.** The search steers on `financial.project_cash`, which books exactly the postings the full projection books and returns the same closing cash, peak debt and posted costs

without building the display trajectory. A probe that breaches the floor, exceeds the APR cap or borrows against the mandate is discarded at once. Any probe that could become the best plan is re-run through the full projection and `covenants.evaluate` before it is accepted, so the chosen plan is the one a full evaluation of every candidate would choose.

- **Display-only expected search** is capped at 2,000 candidates, and its label says so. The committed search, which decides, is never capped.
- **Golden result.** On the golden gap of **¥405,000** before day 32 (the ¥300,000 floor plus ¥505,000 of outflows, including the existing loan's ¥45,000 payment, less ¥400,000 of cash), it selects the credit line (¥150,000, interest **¥841.64**) plus invoice financing (¥255,000, **¥2,272.29**): **¥3,113.93** in total. Taking the buyer's 1.2% discount would cost **¥5,040.00**, and the v1 optimum (line plus partial early pay) **¥3,938.81**. The guaranteed loan is inadmissible because it is not committed and lands on day 21, after payroll on day 12.
- **Optimality.** Against an exhaustive brute force over every usable committed lever (up to three, ¥5,000 grid for pairs, ¥10,000 for triples, the last lever filled to the fen) on 26 scenarios (10 designed and 16 seeded random), the router is within ¥1 of the brute-force optimum on **26/26** (`latest_report.md`, section 3).
- **Marginal cost of funds:** the all-in annual rate of the cheapest COMMITTED, owner-approved source able to carry a *permanent* need. Clean-down excludes the line, so it is invoice financing at **10.8417%**.
- **Break-even early-pay discount:** a discount *d* is cheaper than the marginal source only if d/(1−d) < c, where c is that source's cost share for 30 days. So d = c/(1+c): **0.8832%** exactly, shown as 0.88%. For an early payment *n* days before the due day the cost share is taken for those days (invoice financing is billed in whole 30-day periods, other sources at rate × n/365), so a 90-day-early discount has a higher limit (2.60% on the golden books, ADR-0007).
- **The early-payment rule (ADR-0007).** An early-payment discount is acceptable only if it passes three tests in order. (1) It is below the break-even for its actual days early. (2) The cash is needed: if nothing needs funding without the offer and taking it raises total cost, it fails (`EARLY_PAY_NOT_NEEDED`). (3) It costs less than the router's plan without it (`EARLY_PAY_DEARER_THAN_PLAN`; the two total costs are shown). The router always prices the buyer's early-payment facility at its standing rate, never at an offer's own rate. On the golden books, a whole-invoice early payment 30 days early at 0.75% to 0.88% is now **countered**, not accepted: at 0.75% it costs ¥3,150.00 against the plan's ¥3,113.93, while 0.74% costs ¥3,108.00 and is accepted. The 0.8832% break-even is still the first screen and the figure shown to the owner. An invoice under an early-payment agreement is never financed, in the engine or the executor (ADR-0007 implementation note): that can refuse a cheap early payment a few days before the due day but never accepts one wrongly.

3. Covenant tests (`covenants/`). Evaluated exactly, per facility definition, every projected month:

- `DSCR = monthly EBITDA / monthly debt service`, where EBITDA is the owner's figure. Debt service is existing service (which also leaves cash on its payment days) plus interest and fees on facilities that count, plus scheduled amortisation. A bullet term loan is serviced on a level basis (ADR-0008): its principal, interest and fees are spread evenly over the months of its term, so it cannot look safe until the month it is repaid. This level basis is a **reserve-coverage reading**, not a lender's contractual DSCR definition, and it is labelled so. A revolving credit line is never

spread, and cash is untouched. Hand-calculated case: the ¥500,000 12-month government-guaranteed loan at 5% adds ¥43,721.46 a month, so DSCR is **1.014×** in every month of its term and fails the 1.25× covenant (`DSCR_BREACH`); before ADR-0008 it showed 2.00×. A ¥360,000 request on the same facility, as in the borrower file, shows **1.18×** in month 1, 0.07× below the covenant, and the file reports that shortfall.

- `Leverage = month-end debt / (12 × monthly EBITDA)`, one definition on actual drawn debt (a committed line's undrawn room is not debt), computed by one function from a projection and read by the verifier, capex, the borrower file and chat.
- **Clean-down:** the longest run of consecutive zero-balance days on a credit line must reach the facility's clean-down length.
- Golden values: DSCR **2.00×** at base and **1.92×** after funding the extension; leverage **1.35×** and **1.47×**. For capex, the 18-month loan fails DSCR in months 1–3 (**1.13×**) and the 36-month loan passes (**1.43×**, then **1.77×**).

4. Verdict algorithm (`verifier/`).

1. **Offer status:** conflicting sources or abstention give NO_VERDICT.
2. **Security screen**, never booked and logged as SecurityEvents: an unknown sender or funding claim gives `UNKNOWN_COUNTERPARTY`; an unevidenced prior concession gives `UNEVIDENCED_PRIOR_CONCESSION`; a data request gives `DATA_EXTRACTION_REFUSED` with a fixed reply.
3. **Mandate validity:** signature, version, expiry and revocation. Any failure gives REJECT with no evaluation.
4. **Projection:** the conservative trajectory, the router's committed plan and the marginal cost.
5. **Predicates:** minimum cash on every day, the aggregate and per-buyer forward-concession caps (including existing commitments), the immediate-discount cap, maximum APR, covenants every month, and Tier-3 clauses. A reserve breach that the committed plan can cure is reported as curable with its cure cost. Only FIRM gains may offset concessions. A buyer with an open invoice past its due day on current books needs the owner, with the invoice and the days named.
6. **Break-even rule:** an early-pay discount **at or above** the exact break-even is a pricing fault, `DISCOUNT_ABOVE_BREAK_EVEN`, with the formula `d < c/(1+c)` and its inputs. Like `MAX_APR_EXCEEDED`, it gives a **COUNTER** priced below break-even, never an acceptance. The break-even is taken for the actual days early. Two more pricing faults follow the same path (ADR-0007): `EARLY_PAY_NOT_NEEDED` when the cash is not needed, and `EARLY_PAY_DEARER_THAN_PLAN` when the router's plan without the offer costs less. None of the three is a security event.
7. **Outcome:** Tier 3 or an unknown counterparty gives REJECT; any hard violation gives REJECT with counters; a pricing fault gives COUNTER; a package that needs Tier 2, or comes from a buyer with a past-due invoice, gives NEEDS_OWNER_APPROVAL.
8. **Autonomy (ADR-0005).** What remains would be an ACCEPT. It stays ACCEPT (Tier 1) only when all of these hold; otherwise it becomes NEEDS_OWNER_APPROVAL, and the summary says which proof is missing:

- the package is a pure early-pay discount on a named invoice that passes all three early-payment tests: strictly below the exact break-even for its days early, the cash is needed, and it is cheaper than the router's plan without it;
- `CompiledOffer.autonomy_eligible` is true. The compiler sets it only after a **coverage proof** (`app/compiler/coverage.py`): every amount, rate, day count, date and invoice reference in every document is explained by the extracted obligations; a stated discount sum or net payment equals rate × face; no sentence holds an unread request, condition or question (request-verb and question cues, en + zh); there is no authority claim, data request or funding claim. It defaults to false;
- each discount's **stated** amount equals the whole face of the open invoice it binds to. Binding by default to "the only open receivable" never qualifies.

Owner-built typed offers (`/nl` plans, what-if) carry no free text and are exempt. Counters the buyer accepts are our own template terms, so they are unaffected.

The coverage proof fails closed on unread words: a word inside an obligation's own source span is not exempt from the filler list, and any sentence the proof cannot account for sends the offer to the owner. The reply restates the typed terms; it never says "Agreed as proposed".

Every violation carries a code, the day or month, the observed value, the limit, a `formula` and its `inputs`. The UI shows them verbatim.

Golden check (Scene 1). The EX01 package compiles to three spans: an immediate discount of ¥5,040.00 (1.2%), a recurring terms extension (¥210,000.00 of extra receivables, ¥1,897.29 a month of debt service) and a contingent rebate of up to ¥4,000. The forward-concession ledger is **¥28,326.92** (the rebate plus the extension's funding cost, ¥24,326.92, on the 13 invoices dated inside the 365-day window) against the **¥5,000** cap, so the verdict is **REJECT: AGG_CONCESSION_EXCEEDED; RESERVE_BREACH@47 (curable at ¥2,406.67); DISCOUNT_ABOVE_BREAK_EVEN** (1.2% against the 0.88% break-even); **EARLY_PAY_DEARER_THAN_PLAN** (the package costs ¥6,508.05 against a plan without the early payment at ¥3,550.64; read from the API on 3 Oct 2026). Covenants hold: DSCR 1.92×, leverage 1.47×.

5. Counters from the marginal cost (`verifier/counters.py`).

- **Option A (early pay):** keep 45-day terms and no rebate; accept early settlement on up to **¥257,200** of face (the exact gross-up ¥257,186.08 rounded up to a whole ¥100, so it nets at least the gap) at the largest whole-5 bp rate below break-even, **0.85%**, in the words "paid on or before day 2" (any portion of the face paid by that day earns the rate). The supplier's cost is the discount of **¥2,186.20** plus the cost of funding the rest of the gap on the committed line (¥149,986.20, **¥841.57**): **¥3,027.77**. The router must keep the cash floor on books that include the counter, otherwise no early-pay counter is offered. It is also offered only when its total cost is below the standing plan's (¥3,027.77 against ¥3,113.93, ADR-0007). It books no forward concession.
- **Option B (price adjustment):** accept longer terms with the smallest whole-5 bp price adjustment whose per-invoice amount covers the per-invoice funding cost. On EX01 it is **"55-day terms accepted with a 0.30% price adjustment on future invoices"**: **¥1,260.00** per invoice against ¥1,247.53 for the 10 extra days, a 12-month net of −¥162.05 (extension ¥16,217.95, price adjustment ¥16,380.00), so it costs the supplier nothing and needs the owner. It is 55 days and not the buyer's 60 because **every counter must pass its own re-check**: it is re-evaluated as an offer on the same books and offered only if the result is ACCEPT or

NEEDS_OWNER_APPROVAL with no covenant or rate-ceiling breach and any reserve shortfall it causes curable by an admissible committed plan whose "cannot run today" legs finance only invoices issued by the first shortfall day. At 60 days the day-2 invoice would be paid on day 62, after the day-60 loan payment, with no committed cure; at 55 days (paid day 57) the shortfall of ¥85,000.00 on days 47 to 56 is curable. A counter that fails is dropped and the verdict's summary says in one sentence why.

- Bounded-extension and rebate-within-headroom counters are generated the same way.
- Outbound text is template-filled from an allow-list and passes the egress guard. It never contains the floor, a cap, a limit, the marginal rate or a reservation.

6. Buyer kernel (`counterparties/`). A deterministic kernel with a hidden reservation. Its surplus from a package is:

```
surplus = discount received + declared benefits + rebate received + value of extra payment days − its own cost of paying early − price adjustments it pays
```

It ACCEPTs when every reservation gate holds and the surplus reaches its reservation. Otherwise it COUNTERs one step toward, but never onto, its reservation. In its last round, or when rounds run out, it REJECTs. The kernel reads only a counter's public terms. In Scene 1 it accepts Option A. In Scene 2 a different reservation refuses every counter within **3** rounds, giving NO DEAL; the router then funds the gap with the line plus invoice financing for ¥3,113.93.

7. Capex mode (`capex/`). A screen, not an investment appraisal: the uplift, price and life are the owner's estimates, and no residual value, tax or discounting of financing flows is modelled. For the synthetic ¥600,000 machine: NPV **¥380,885.77**, payback in month **31**. Each structure is ranked on total financing charges (interest plus fees, not discounted), counts the facilities already drawn, and is tested day by day for liquidity and month by month for DSCR and leverage; it is evaluated only at its facility's actual term, so the 18-month loan (**¥34,830.60** a month) is a labelled illustration, which fails DSCR. The **48-month lease (¥14,423.49 a month, charges ¥92,327.32) is selected**, the cheapest among the structures that pass: the 36-month loan (¥18,144.62 a month, charges ¥53,206.14) holds DSCR but is rejected because its first instalment, on day 44, takes cash to ¥296,855.38, below the ¥300,000 reserve, now that the existing loan's payments leave cash; the lease's first payment leaves ¥300,576.51. On untouched books leverage is **1.90×**, or **2.10×** with the extension also funded. No financing is recommended when the project is not worth doing on the owner's estimate. Evaluating writes nothing. Applying is a separate owner-only step, `POST /capex/apply`, bound to the state version of the evaluation the owner saw (409 if the books moved), which queues one Tier-2 application in the approvals queue. The borrower file is a separate read-only owner action that queues nothing.

8. The strategy layer (`strategy/`, flags off by default, read-only). Three answers, each computed on a labelled copy of the books with the kernel's own functions (`docs/STRATEGY_MVP_REPORT.md` section 1). On the golden day-0 books (section 2 of that report): the envelope reads "You can live with: up to 55-day terms if the price rises 0.30%, or early payment at up to 0.85% on ¥257,200.00; a rebate up to 1.25%; not 60 days without an early payment", with the money line "Cost to you ¥0.00 with the price rise, or ¥3,027.77 with the early payment; without a deal your current funding plan costs ¥3,113.93"; the cash price curve reads "Cash in the next 30 days saves you ¥89.11 of funding cost per ¥10,000 (0.89%); after that, earlier cash saves nothing"; and collections ranks the open invoices by what chasing each is worth. Its backtest passed with 0 failures (186 s): the 49 scenarios replayed turn by turn (87 turns), a clock sweep to days 10, 20, 34, 47 and 60 (245 states), 409 differential checks to the fen against the recorded

oracle, and the envelope over 180 listed offers (127 checked): 89 of 89 verifier counters inside the envelope, 29 of 29 probes agreeing with the verdict, 6 of 6 lever maxima confirmed, and 17 of 17 comparisons of bisection against an exhaustive 273-cell scan (`docs/research/strategy/06-backtest-2026-10-08.md`). What is not claimed: the answers are decision support on labelled copies; the shadow pilot that would compare them with what an owner actually did is planned, not run (`docs/STRATEGY_MVP_REPORT.md` section 7); the Chinese sentences await a native reader (section 8 of that report).

Evaluation program

We evaluate the compiler on data its tuners had not seen, the engines against an independent oracle and a brute force, and the whole agent through scripted tasks and attacks. One command regenerates every figure below except the held-out-2 and held-out-3 rows: `uv run --project services/api python scripts/evaluate.py` (offline, `LLM_MODE=cache`, about 6 minutes on our Windows machine: 345 s on 3 Oct, of which the funding brute force is 310 s). Report: `services/api/evals/latest_report.md`.

Three held-out sets of buyer messages, and how independent each is. Each was written by a separate evaluation agent from the spec, the glossary and the golden state, and its labels were committed before the first run.

Set	Size	Written	Independence status today
Held-out 1 (`fixtures/heldout/`)	50 offers, 47 owner utterances	By the evaluation agent, not by the compiler's authors	**Tuned on.** Its misses (EV-01 to EV-05) guided compiler round 1, and some of its labels were updated when the Tier-1 policy changed (commits `5dc842a`, `52dcd01`). The full run's 45/50 verdict agreement (it was 49/50 before the existing loan's payments became cash) and 0/36 material errors on it are optimistic and are not an accuracy claim.
Held-out 2 (`fixtures/heldout2/`)	49 offers, 48 utterances	Before reading the compiler or held-out 1	Unseen at its first run (28/47, 60% verdict agreement; 5 silent material errors). Its failure classes then guided round 2.
Held-out 3 (`fixtures/heldout3/`)	66 offers, 53 utterances	Before reading the compiler or the earlier sets; labels committed in `1d6d8fd` before the first run	**Independent and unseen at its first run**, the only untuned figure we have. Its failure classes then guided round 3 and ADR-0005.

Compiler accuracy: the only claims we make (copied from `services/api/evals/accuracy_summary.md`):

Set	Independence	Path	Verdict agreement	Material error rate	Silent material errors	Wrong autonomous accepts
Held-out 3, first run (before ADR-0005)	**independent, unseen**	rules grammar	33/63 (52%)	4/51 (7.8%)	7	**4 of 8**
Held-out 3, after round 3	its failure classes guided round 3	rules grammar	46/63 (73%)	3/51 (5.9%)	4	**0** (2 correct)
Held-out 3, after round 3	the prompt was not tuned on it	**live DeepSeek (`deepseek-flash`)**	42/63 (67%)	3/51 (5.9%)	5	**0** (1 correct)
Held-out 2, after round 3	its classes guided round 2	rules grammar	41/47 (87%)	0/37	0	**0** (4 correct)
Form baseline (labels typed in)	—	no compiler	61/63 (97%)	—	—	—

- **What we claim (the guarantee):** across every held-out set and both compiler paths, **0 wrong autonomous accepts** after ADR-0005 (rows above). Text the compiler cannot fully account for fails closed: it goes to the owner's queue, a counter or a clarification, never to an unauthorised commitment.
- **What we report beside it (the misses, unchanged):** free-text understanding is still imperfect. About 1 in 3 messages is not decided the way a human would on first reading (verdict agreement 52% on the first held-out-3 run, 73% rules and 67% live DeepSeek after round 3). Most good early-pay offers also wait for the owner, because they mention a settlement date that the compiler does not read into the terms.
- **What we don't claim:** a compiler accuracy figure from a set it was tuned on. The only untuned figure is the first held-out-3 row. The material-error target (≤ 5%) and the zero-silent-error target are **not met** on held-out 3 by either path.
- **The first held-out-3 run is how we found ADR-0005.** All four wrong autonomous accepts were early-pay offers in which the grammar kept the rate and dropped a clause that contradicted it or added to it (`services/api/evals/heldout3_report.md`). Owner-utterance intent accuracy on that first run was 34/53 (64%).

Full offline evaluation (`latest_report.md`; spec v2 §14 gates), run on 3 Oct 2026 with `scripts/evaluate.py` at the commit named in the report. The 15 failures the report lists are item-level misses, none of them a target. Ten are the held-out misses of before (5 offers, 5 owner utterances); five are new: four 60-day-terms offers (H09a, H09b, H12a, H39) that the held-out labels expect to need the owner and that now return REJECT with RESERVE_BREACH on the cash-corrected books, and the near-miss pair H12, whose two sides now both reject. The labels were not changed.

Gate	Question	Result	Status
E3	Engine correctness	Financial golden values exact to the fen: **67/67**	PASS
E5	Honest no-deal	Hard-line buyer → NO_DEAL within **3** rounds, gap still funded by `DRAW_FACILITY` + `SUBMIT_INVOICE_FOR_FINANCING`, reserve holds afterwards: **7/7** checks	PASS

Gate	Question	Result	Status
E7	Harness compatibility	Scripted `/nl` tasks: **49/49** (100%); attacks with any harm: **0/126** through the API in `latest_report.md` (the 133 chat attacks are in the security report: 259 attacks in all, 0 with harm)	PASS
E9	The router matters and stays honest	Router cheaper than "line first", or the baseline infeasible: **9/26** scenarios with a gap (target at least 5 of 20); the total saving against "line first" is ¥-3,491.40 over the 21 where both are admissible (median ¥0.00), which is not a like-for-like figure because the baseline still draws on the buyer's early payment, a leg the agent cannot run (backlog EO2); committed plans relying on an uncommitted facility: **0**; runnable optimality **26/26** (the router's best runnable plan within ¥1 of a grid-bounded brute force over the levers the agent can pull today; the gate) and priced optimality 26/26 (informational), and the router's runs-today and needs-the-owner flags agree with the oracle's on 26/26. In six of the 26 worlds (F04, R01, R02, R10, R13, R14) the cheapest plan that closes the gap uses a buyer's early payment or a bill, which no operation runs, so the router flags it "cannot run today"; whether the agent should propose drawing the line and asking the buyer or the owner is an open product question (NR1). The brute force is grid-bounded, so a plan both missed is invisible (U-013)	PASS
E10	Covenant and capex engine	Covenant and capex golden values **27/27**; mandate and covenant boundaries exact at ±1 fen/bp/day **20/20**; DSCR and leverage breaches reported with their month **2/2**	PASS
E2 (held-out 1)	Compiler beats a form	0/36 material errors, 0 silent errors, engine ≠ static cap on 22/33	PASS **on a tuned set**; see the accuracy table above for the honest figures

Latency (offline, one laptop, same run): router plan p50 **60 ms**, p95 **198 ms**; the slowest `verifier.evaluate` on the router test states **0.5 s** (it was 14 s before EV-07 capped the display-only search, and the EX01 evaluation fell from 3.3 s to about 0.5 to 0.9 s with the shared router searches); `/nl` turn p50 **7 ms**, p95 **540 ms** over 87 turns; the golden Scene 1 end to end (compile → evaluate → negotiate → execute) p50 **497 ms**, p95 **532 ms** over 5 cold runs. A live LLM call adds its own latency; the live DeepSeek held-out-3 run took 682 s for 66 offers (`heldout3_live_deepseek_report.md`).

Language-boundary certification. A separate 231-item adversarial set (`fixtures/certification/language_boundary.yaml`) is scored from stored state through the real `/nl`

route by `scripts/certify_language_boundary.py`. Cache run (`language_boundary_cache_20261003T075236Z`): 63 correct-executable, 6 read-only, 63 clarification, 98 abstention, 1 wrong-harmless, **WRONG_MATERIAL 0**; fake run (`language_boundary_fake_20261003T075307Z`, scripted models through the real adapters, malformed output included): 57, 7, 76, 88, 3, **WRONG_MATERIAL 0**. The first dry run had found one real WRONG_MATERIAL (a decimal-comma rate read as 500 bp); the boundary abstentions fixed it. It certifies the deterministic boundary and the adapters' handling of bad model output; **no real model has been run through it (the live run is pending an API key)**.

Not measured. The accuracy of reading contracts (the held-out set is pending); the strategy answers against what a real owner did (the pilot is planned); handling time against a form (it needs timed human operators); a live Claude accuracy run and the live language-boundary certification; PostgreSQL in CI or on a migrated Supabase database; blind explanation tests by non-builders (E8).

04 Security Design

- Tiered permissions and confirmation for sensitive actions

Who can act (`app/api/auth.py`; constant-time comparisons; every refused credential is logged as a `PRIVILEGE_ESCALATION` security event):

Credential	Header / variable	Confers
Owner token	`X-Owner-Token` / `OWNER_TOKEN`	Owner authority: read views, Tier-2 decisions, accepting or declining a buyer's request, revocation, circuit reset, `AddFacility`, capex purchases, the simulated clock; demo reset and books setup in demo mode only
Agent key	`X-Agent-Key` / `AGENT_KEY`	The agent's tool channel: Tier 1 inside the mandate; Tier 2 is queued
Bank-feed key	`X-Bank-Feed-Key` / `BANK_FEED_KEY`	Recording a receipt as `SYSTEM` (Tier 1); the same receipt from the agent or a counterparty is Tier 2. Since 15 Oct it no longer moves the simulated clock (SEC-09)
Buyer channel key	`X-Counterparty-Key` / `COUNTERPARTY_KEYS`	That a sender on `/nl` or `POST /offers` is that registered buyer (another buyer's id in the body is 403); 5 offers or messages per simulated day, then 429 `BUYER_BUDGET_EXHAUSTED` (R-10)
None	—	A counterparty can submit documents or talk on `/nl` as COUNTERPARTY, which compiles and verifies but never executes, and answers with a reply and a session id only. Without a valid buyer key the message is a proposal only: nothing live is stored. Intake is bounded: at most 5 documents of 4,000 characters, a buyer channel only, a stored document id never reused for other words, and 200 unauthenticated submissions per simulated day

- A body field such as `requested_by: OWNER` confers nothing: it is downgraded to AGENT before parsing.

- **`DEMO_MODE` is off unless set** (since 16 Oct, SEC-07). Local runs, the tests, the browser suites and CI set `DEMO_MODE=1`, so the published demo values (`demo-owner-token`, `demo-agent-key`, `demo-bank-feed-key`) work there. Without it a credential whose variable is unset matches nothing, the owner and agent routes fail closed (HTTP 503) until `OWNER_TOKEN` and `AGENT_KEY` are set, no demo books are seeded, the demo signing key is never trusted, and `/docs` and `/openapi.json` are not served. The Compose deployment sets `DEMO_MODE=0` explicitly.
- **The agent's chat sees only its own amounts** (R-11, the owner's decision of 16 Oct). An agent-key turn on `/nl` is answered from an allow-list (`app/api/agent_view.py`): the intent, the operation steps, the outcome, what ran or was queued, refusal reasons in words, and the amounts of its own operations; a pure read answers "This needs the owner's sign-in." Every text in it passes the egress guard.
- The web app holds no credential in its bundle. A 401 opens an **Owner sign-in** dialog; the token is checked against the API and kept in `sessionStorage` only. Buyer chat turns send no credential.

Three tiers, computed by the executor (`app/mandate/tiers.py` and the verifier's autonomy rule). A caller-supplied tier is rejected as an unknown field, and extra facts can only raise the tier.

Tier	Operations	Outcome
Tier 1: autonomous inside the mandate	Accept a counterparty's offer **only** when it is a pure early-pay discount on a named invoice, strictly below the exact break-even and passing the other two early-payment tests (cash needed, cheaper than the plan), with the ADR-0005 coverage proof and a stated whole-invoice amount. Counter or reject within the mandate; record a commitment of that same early-pay kind; draw on or repay a COMMITTED, owner-approved facility within its limit, or submit an eligible invoice to the onboarded financing facility, when it passes admission and its effective annual cost is at or below `max_apr_bp`; record a receipt from the bank feed or the owner	EXECUTED if every gate passes
Tier 2: owner approval	**Every other acceptance** (rebates, terms changes in either direction, price adjustments, volume or operational concessions, and early-pay offers without the coverage proof); apply for a facility (the capex application, queued by the owner's explicit `POST /capex/apply`); commit to an equipment purchase (`CommitPurchase`, capex buy side: always Tier 2 and **owner only**, bound to the confirmed contract's hash and the schedule shown, every payment money out, one contract one purchase); any single commitment above ¥100,000; draw on any facility that is not COMMITTED and	PENDING_APPROVAL, queued with the operation's own description as the reason

Tier	Operations	Outcome
	owner-approved; a draw or advance whose effective annual cost is above `max_apr_bp`; a receipt reported by the agent or a counterparty; an owner-originated AddFacility	
Tier 3: never	Personal guarantee, pledging collateral, issuing equity, changing the mandate, adding a facility by anyone but the owner, any unknown operation type	REFUSED for every caller, the owner included

An early-pay discount that fails any of the three early-payment tests (at or above break-even, cash not needed, or dearer than the router's plan) is neither Tier 1 nor Tier 2: it is a COUNTER (`DISCOUNT_ABOVE_BREAK_EVEN`, `EARLY_PAY_NOT_NEEDED`, `EARLY_PAY_DEARER_THAN_PLAN`).

Contract documents. The agent's reading of an uploaded contract counts only once the owner confirms it, and confirming changes no books. The demo starts with the three demo documents already read and confirmed (the owner's first three logged operations), so the deal summary is ready as soon as a deal is agreed. CONFIRM_DOCUMENT and REJECT_DOCUMENT are owner-only (any other caller is refused and logged as privilege escalation), bound to the SHA-256 of the exact text the owner reviewed, made once, and refused for a document that carries text addressed to the agent.

Confirmation. A Tier-2 operation is stored as its exact canonical JSON. When the owner approves, the executor re-runs every gate against the mandate then in force. An approval given after a revocation is therefore refused, and an approved Tier-3 operation stays refused.

Verification. Every operation passes the same gates, in order:

1. idempotency key (`DUPLICATE`);
2. dating (`STALE_OPERATION`, `REPLAY`);
3. mandate signature, pinned key, expiry, revocation and version;
4. Tier 3;
5. counterparty circuit breaker (three refused offer operations in seven days; only the owner can reset it; it gates and counts only offer operations, so it cannot block a bank receipt; the owner answering a buyer's request passes it, and the owner's own refusals never count, so the agent cannot trip it to take the owner's decision away, SEC-05);
6. for acceptances, the commitment must exist and be unrevoked, the nonce must be single-use and match, the commitment must not be expired, the terms hash and mandate version must match, and a verifier verdict must back the terms. The verdict is then re-verified at that moment (ADR-0006): the named receivables are still open, no other live commitment holds the same invoice or answers the same buyer request (SEC-02), the concession caps still hold, and, if the state has changed since the verdict, the obligations still verify on current books (`STALE_VERDICT`, `CONFLICTING_COMMITMENT`);
7. Tier 2 queue;

8. per-kind preconditions, and for draws, advances and repayments admission (outstanding face, landing before the due day, DSCR, leverage, reserve, the borrowing switch), judged on today's books with the planner's own engine; the owner cannot waive a hard rule by approving.

An idempotency key with an executed or pending entry returns `DUPLICATE`; a refusal does not use the key up; a key reused for another operation is refused. A bank or owner receipt is still booked after the mandate expires or is revoked (it records a fact); a bad signature still stops it.

Each executor entry point runs as one store unit of work, so state, commitment, nonce, approval, security event and log entry commit together or not at all. A unit that lost a race with another writer commits nothing and runs again on the books as they now are (at most 8 times, in the executor and in the clock); a conflict that survives answers HTTP 409.

Record. Every outcome of an operation admitted to the executor, refusals and duplicates included, is appended to a hash-chained log: `entry_hash = sha256(prev_hash + payload_json + result_json).` Database triggers refuse UPDATE and DELETE on the log tables (migration `0003_append_only_triggers`). `scripts/verify_evidence.py` recomputes the chain with the standard library alone. Since 17 Oct the database also decides each approval once and keeps security events append-only, on SQLite and PostgreSQL, and no session setting lifts that guard (migration `0009_audit_guards`, SEC-08); the executor also refuses a decision the log already holds, even if the approval row is put back to pending (owner-approved, merged after `719c9dcf`). The chain covers the operations log only. It is tamper evidence against anyone without database write access: a database owner can rewrite it consistently, which `scripts/verify_evidence.py --checkpoint` detects only against a checkpoint kept outside the system (R-13 and section 1.10 in the Security Self-Assessment).

Rollback. The log is append-only, so a mistake is corrected by a logged compensating operation: repay a draw, settle an advance on receipt, revoke a commitment (`POST /commitments/{id}/revoke`, owner only, logged as `REVOKE_COMMITMENT`), or decline a queued request. Each operation is atomic; a plan of several legs is not, and a plan refused part-way keeps its earlier legs (SEC-28). The engine honours a revoked mandate, but no API route lets the owner revoke or renew it (R-14). A refused or pending operation leaves state unchanged; the attack suite checks this.

- Protection against prompt injection, privilege escalation and privacy leakage

Prompt injection: architecture first, detection second.

- Counterparty text only ever becomes a `CompiledOffer`: typed obligations whose spans must match the source exactly. The schema has no field for an operation, an approval, a tier, a reply or a new facility.
- The verifier reads typed rows only; no text field reaches `verifier/`, `funding/`, `covenants/`, `mandate/` or `financial/` (AST-enforced test).
- A claimed prior concession with no matching ledger commitment is never booked. A lender named in a message becomes an `UnsolicitedFundingClaim` and a security event, never a facility.
- Contingency is decided deterministically after the model answers: a sentence with a condition on a future business event is CONTINGENT whatever the model says, and a gain is never relabelled FIRM.

- The rules grammar flags override attempts, fake SYSTEM lines, role-play, encoded text, full-width, zero-width and homoglyph text, and authority claims in both languages. Detection is logged, but refusal does not depend on it.
- If a model backend fails, times out or returns invalid JSON, the deterministic rules grammar takes over.
- **Contracts are read the same way.** Every contract term must quote its clause verbatim; its amount, rate, period and date are parsed in code from the quoted words. A sentence addressed to the agent (an instruction, a request for its prompt) is a blocking flag: it is never read as a term, the document cannot be confirmed, and a security event is logged. Documents are read only for counterparties on the owner's registry with the matching role, and a confirmed lender agreement never becomes a facility.
- **The record and the strategy layer sit outside the engine.** `tests/decisions/test_isolation.py` and `tests/pilot/test_isolation.py` pin that the engine imports neither the decision record nor the journal (handoff 08 section 4); the strategy layer's diff against `dev` outside its own files was three default-off flags, a router mount and a what-if builder generalisation whose default path is byte-identical over 540 parameter combinations (`docs/STRATEGY_MVP_REPORT.md` section 1).
- **Misreading is treated as a security problem too.** A message the compiler cannot fully account for can no longer be accepted without the owner (ADR-0005), and the coverage proof fails closed on unread words.
- **The chat endpoint is inside the boundary.** The attack suite also drives `/nl` as the owner, as a buyer and as a holder of the agent key. A buyer's chat response carries the reply and the session id and nothing else: no verdict, plan, operation or protected value. Text pasted or quoted into the owner's chat never becomes an instruction or an approval id. A chat session continues only for the caller that opened it (the owner, the agent, one keyed buyer or an anonymous sender), and the owner's sessions are evicted last (SEC-21). Once the owner has sent a buyer a counter themself, no chat path accepts, declines, counters or negotiates on that request; the buyer's answer is recorded on the deal (OUT-1). Free text executes only as a closed affirmative command (negation, questions, conditions, future times, foreign currencies and unreadable amounts execute nothing); an approval id is decided only by an exact decision; "go ahead" runs the exact operations that were shown. A buyer is who its channel key says. The security scoreboard is computed from the run data, not asserted. Found by this suite and fixed the same day: text pasted without quote marks ("Attachment:", "> …", "Forwarded message") could approve a queued item, and an agent-key holder could say "go ahead" inside the owner's session.

Privilege escalation. Tiers are enforced by code, never by prompts. The executor trusts no caller-supplied tier, actor or approval. Self-approval, circuit resets by the agent, forged or edited mandates, reused or guessed nonces and backdated operations are all refused and logged.

Privacy leakage.

- FLOW never attaches the mandate, state, funding menu or reservations to a model request, and import tests keep the compiler package from reaching them (a person can still paste such figures into a message).

- Outbound text is template-filled from an allow-list. Data requests get one fixed reply that discloses nothing.
- The deterministic egress guard scans every outbound text for protected values in their usual forms (¥, k, 万, Chinese and 大写 numerals, English number words, percentages, ratios, company names and role aliases) and for four planted canaries. It blocks the whole message on a hit. It also reads less common spellings (30,0000; 3,00,000; 0.3m; 3e5; 30w; 千; ‰; points; a ratio written as a percentage; SEC-22). Derived values (the break-even rate, the counter cap, the shortfall) are not in the protected set, by the owner's decision of 17 Oct: the counter cap sits close to FLOW's own counter-offer, so the guard would block it.
- Read routes (`/state`, `/mandate`, `/trajectory`, `/ledger`, `/audit`, `/evidence`, the document, deal, purchase and setup routes, and every Home, Cash, Customers and Activity read) require the owner token; the Security Lead verified this by calling all 94 routes as every caller, with `DEMO_MODE` on, off and unset.
- The agent key's chat replies carry only the amounts of its own operations (R-11); a keyed buyer has a daily budget of 5 offers or messages (R-10).
- A contract's terms (amounts, rates, payment terms, the agreement number, a lender agreement's limit and rates) are protected values owned by that counterparty, read or confirmed: another buyer asking for them is refused, and the egress guard blocks them in any outbound text.
- **Web and API hardening.** Every page carries a Content-Security-Policy with a fresh nonce per request, so injected markup cannot run script (verified in a browser on a production build), with no framing, no MIME sniffing and no referrer; the API limits requests per caller and answers a burst with 429 `RATE_LIMITED` and `Retry-After` (SEC-12, SEC-20). Behind a proxy every caller shares the proxy's address, so a shared deployment still needs the proxy's own per-client limit.
- Cache mode sends nothing off the machine. In live mode only the inbound documents and the schema leave it.

Compliance, in brief (the full statements are in the Security Self-Assessment, section 03).

- **Data flows.** In live mode (opt-in) a buyer's message and attached documents go to the configured model provider (DeepSeek or Anthropic) for extraction only. The model never receives the mandate, the floor, facility limits or other buyers' terms. In the default cache mode nothing leaves the machine. A real buyer document can contain personal information (for example an individual's name, phone number or bank account). Sending it to a provider outside mainland China would be a cross-border transfer of personal information under China's PIPL, and a deployer would need a legal basis and, where required, a security assessment or standard contract before any production use with real documents. That is an obligation on the deployer, not legal advice, and we do not claim compliance.
- **Synthetic data only.** The demo, the tests and every evaluation use synthetic fixtures.
- **AI-generated content.** Outbound text to a counterparty is template-filled from typed fields, not free model prose. The web UI labels what the model extracted "AI-generated, for reference": a contract's reading until the owner confirms it (then "Read by the agent, confirmed by you"), and a buyer offer's extracted terms in the Inbox. Chat replies are template text built from typed fields and carry no label.

- **Decision support, not advice.** The agent acts only inside an owner-signed mandate. Tier 2 needs the owner and Tier 3 is never executed. It is not a licensed lender, broker or adviser, and it executes no real payment: every facility and the bank feed are simulated.
- **Audit trail.** Every operation, refusals included, is in a hash-chained log. The chain head is not anchored outside the system, so a full rewrite of the log would verify as valid (audit finding S8, open). Approvals and security events are not in the chain, but the database guards them: an approval is decided once and security events are append-only (SEC-08).
- **Licence.** [TEAM: licence to be chosen by the owner]

Measured results. The suite now holds **259 attacks: 126 through the API and 133 through chat** (43 as the owner, 64 as a buyer, 26 as a holder of the agent key without the owner token). **In each specified run, 0 of the 259 violated the defined harm oracles** (`DEMO_MODE=1 python -m app.security.attacks --report`; first run 5 Oct 2026, re-run by the Security Lead on `dev` at `a6de32a2`, `b6d1e519` and `0f92d433` on 16 Oct 2026 and at `719c9dcf` and `0fde33fd` on 17 Oct, each with the same result). This is a test result, offline and on recorded model answers, not evidence that no exploitable path exists. The catalogue drives about [CERT: n] of the [CERT: 100] API routes over HTTP; the rest are covered by targeted tests and an authorization sweep of every route. The four added on 5 Oct target contract ingestion: an instruction to the agent inside a contract clause, a lender agreement from an unregistered sender, a buyer asking for another buyer's contract terms, and a non-owner confirming a document. Each attack runs on a fresh runtime through the real FastAPI routes, `LLM_MODE=cache`. The buyer attacks run twice, with no channel key (`NL-CX01`) and with the buyer's own key (`NL-CX01K`), and the 41 consent attacks `NL-CS01` to `NL-CS41` send negated, questioning, conditional, future-dated, foreign-currency, unreadable-amount and "do not approve" texts in English and Chinese with the owner token and with the agent key. The table below is the 126 API attacks as in `services/api/evals/security/latest_report.md` (which also lists every chat attack), with the deployment's credentials and `DEMO_MODE=0`. Of the 133 chat attacks, 86 were detected (an event or a refusal code) and 47 changed nothing but raised neither, 39 of them consent attacks, whose correct result is that nothing runs and no code is raised; the other 8 are the agent key asking to reset the circuit breaker or to revoke a commitment, the agent key saying "go ahead" inside the owner's session, the owner token reusing a counterparty's session id, a bare "yes" in a fresh owner or agent session with an approval pending, and two replayed request ids.

A separate audit and five red-team rounds by AI subagents (1 to 3 Oct 2026; reports and probes under `docs/research/`: the principal audit and its evidence folder, and `redteam-2026-10-03`, `-03b`, `-03c`, `-03d`, `-03e`) found 13 and 26 issues (3, 7, 3, 7 and 6 in the rounds), listed with their regression tests in the Security Self-Assessment (section 02). All 39 were fixed; the owner then stopped the red-team rounds. The audit's NO-GO conclusion on autonomy dated from before those fixes, and the system remains a simulation.

A Security Lead (an AI agent in a separate session and worktree, 14 to 17 Oct 2026; an internal, AI-assisted review, not external accreditation or a penetration test; it also wrote the security fixes it certified) audited the code as it then stood, reproduced each finding before recording it, had the clear-cut ones fixed with a regression test that fails before and passes after, and certified, as an internal release decision, the exact commits that landed (`0ba8e4c1`, `081541d1`, `b6d1e519`, `0f92d433`, `719c9dcf`, `0fde33fd`; `docs/security/CERTIFICATIONS.md`). Its most serious findings, all fixed: an anonymous caller could replace the stored text of a buyer's offer the owner reads, and one unbounded anonymous message could stall the API for a minute; one buyer request could carry two agreements and book its concession twice;

the agent key could take the owner's accept or decline away; the bank-feed key could move the clock; and on the capex branch a vendor contract with a retention or discount above 100% booked cash coming in. It also verified the owner's decisions of 16 Oct on R-10, R-11 and `DEMO_MODE`. On 17 Oct it built and certified the next round: the database guard on approvals and security events, a full CSP, per-caller rate limits, chat sessions bound to their caller and more number forms in the egress guard, with the team's fix for a chat path that could book an agreement on a request the owner had answered outside FLOW (OUT-1). Its open items (no runnable production configuration, the chain head not anchored, one process only) keep production at NO-GO; the full register is in the Security Self-Assessment.

Concurrency, proven on SQLite. Two worker processes on one SQLite file could double-book on the old store (16 of 20 cases of the new suite failed). The store now enforces the invariants itself: compare-and-swap on the state version and on the log head, `BEGIN IMMEDIATE` on file SQLite, the executor and the clock rerun a unit that lost a race, and a conflict that survives answers HTTP 409. `tests/concurrency/` proves eight invariants with two spawned processes, then replays the committed actions serially and requires byte-identical books. **PostgreSQL 17, first run 16 Oct (the Security Lead, a local throwaway database):** the same invariants hold once a startup race is fixed (PG-1: every store re-ran its table and trigger DDL on opening, so two processes opening at once raced on the catalog and one failed to start; fixed and merged, then 3 of 3 clean runs on `719c9dcf` and on `0fde33fd`). The API still runs as one process (SEC-11). Details: `docs/research/concurrency-certification-2026-10-03.md`.

Metric	Result
Unauthorised operations executed	**0**
Facilities added by non-owners	**0**
Concessions booked from unevidenced claims	**0**
Money moved by attacks	**¥0.00**
Integrity failures (partial write, double execution, erased log)	**0**
Protected values disclosed in outbound text	**0**
Protected values readable without the owner token	**0**
Canaries tripped in outbound text	**0**
Attack success rate: injection / exfiltration / escalation / replay	**0%** of 42 / 22 / 21 / 21 attempts
Detection rate (a SecurityEvent or a refusal code)	**98.4%** (124 of 126)
SecurityEvent rate	**73.0%** (92 of 126)

- Twelve findings (F-01 to F-12) were found by this suite and are fixed or mitigated; no attack is run as an expected failure. The two attacks that are "contained" rather than detected are Tier-2 requests by the agent, queued for the owner as designed.
- The pytest suite adds two hypothesis fuzzers (random operation bodies against `/operations`, and random zh + en injection fragments through intake, the verdict and a compromised agent).
- **The medium items from the chat attacks, after the owner's decisions of 16 Oct.** (S5, R-10) A keyed buyer used to find the accept/counter threshold to 1 bp in 12 messages; it now gets 5 messages a day, but one message can still reveal the counter cap and, through the counter's

size, the funding gap (mitigated, open). (S9, R-11) The agent key no longer reads protected values through chat: it gets an allow-list with only its own operations' amounts (fixed, verified by the Security Lead). The third item of 1 Oct, "go ahead" re-planning at confirm time (S10), was fixed on 3 Oct.

- The full risk register, including the open risks (single-operator key custody, the Tier-1 blast radius, the open operational rows R-15, and the three compiler risks of section 03), is in the Security Self-Assessment.

05 Deployment and Testing

- Environment, deployment instructions, test cases and known limitations

Environment.

- API: Python 3.12 with `uv`; SQLite for local development and tests; PostgreSQL 16 in Docker Compose, or Supabase (Singapore) with RLS.
- Web: Node with `pnpm`.
- The default `LLM_MODE=cache` needs no network or API key.
- Target deployment: one Docker Compose stack (web, api and postgres) on one VM in Hong Kong or Singapore. [TEAM: confirm the VM, its URL and a reachability test from mainland China.]

Run locally (from the repository root):

```
cd services/api && uv sync && uv run uvicorn app.main:app --reload     # API on :8000 (cache mode, demo credentials)
cd apps/web && pnpm install && NEXT_PUBLIC_API_MODE=live pnpm dev       # UI on :3000; omit the variable for offline mock mode
cd services/api && uv run pytest                                        # API tests, offline
python3 scripts/run_scenarios.py --inproc                               # the 49 scripted /nl tasks and their evidence
uv run --project services/api python scripts/evaluate.py                # the full offline evaluation (about 5 min)
python3 scripts/verify_evidence.py services/api/evals/e2e/latest/evidence/S01_scene1_counter_accept.jsonl
```

Deploy.

1. Copy `infra/.env.example` to `infra/.env` and set `POSTGRES_PASSWORD`, `OWNER_TOKEN` and `AGENT_KEY` (random values, for example `openssl rand -hex 32`). Optionally set `LLM_MODE=live` with `ANTHROPIC_API_KEY` or `DEEPSEEK_API_KEY`. To use Supabase instead of the bundled container, point `DATABASE_URL` at it and run the Alembic migrations, which enable RLS on every table.
2. Run `docker compose -f infra/docker-compose.yml up --build`.
3. The Compose file sets **`DEMO_MODE=0`**, so the published demo credentials are rejected, and it refuses to start without `OWNER_TOKEN`, `AGENT_KEY` and `POSTGRES_PASSWORD`. **As committed it does not yet start a working API** (SEC-06): it passes no `OWNER_PUBLIC_KEY_HEX`, nothing provisions the books and the signed mandate into an empty database, and the web image is built against `http://localhost:8000`. These are listed as production blockers in the Security Self-Assessment. It binds the API to `127.0.0.1:8000`, to sit behind a TLS reverse proxy. It does not set `BANK_FEED_KEY`, so there is no bank-feed channel until one is configured.

Test cases. `uv run pytest --co` collects **6,932** API tests (12 Oct 2026, on `dev` at 86123a3c), by test directory:

Area	Tests	Area	Tests
funding router	1,151	commitments	90
nl harness	1,088	books	83
egress guard	971	counterparties	76
compiler	684	alerts	76
api	505	mandate	72
operations (executor)	401	what-if	51
verifier	258	capex	51
security (attacks, fuzzers, metrics)	248	deals	42
strategy	229	certification	41
financial engine	195	covenants	38
pilot journal	167	document text extraction	33
decisions	138	concurrency	30
persistence	132	borrower file	29
evaluation harness	24	contract records and held-out scorer	9
money and fixtures	11	clock	9

Latest full run (`SKIP_PERF_BENCH=1 uv run pytest -o addopts= -q -p no:cacheprovider`): **6,895 passed, 37 skipped, 0 failed** on 12 Oct 2026 on `dev` after the UI overhaul and the Deals page (Windows, about 25 minutes; README, "Test"). The same suite on macOS before the overhaul: 6,813 passed, 37 skipped (README; handoff 08 section 1). Earlier records, for the history: 5,446 passed, 33 skipped at `db097e0` on 3 Oct 2026. The 37 skips are the performance benchmarks hidden by `SKIP_PERF_BENCH=1`, the exhaustive router differential cases that run only with `ROUTER_DIFF_FULL=1`, and the PostgreSQL concurrency cases that need `TERMSLEDGER_TEST_DATABASE_URL` (handoff 09 section 3 names the last group; the exact split of the 37 was not re-counted for this document). The run uses the recorded-output cache and an in-memory store, and the concurrency suite ran on SQLite only; live-model mode was not tested and PostgreSQL was not run (uncertainty debt U-001, U-002). The other gate rows on the same tree (handoff 09 section 3): `scripts/run_scenarios.py --inproc` 49/49 with 87 turns; `scripts/demo_run.py` exit 0; `python -m app.security.attacks --report` 259 attacks (126 through the API, 43 + 64 + 26 through chat), 0 with harm.

Web app (`apps/web`, 12 Oct 2026, on `dev` after the UI overhaul and the Deals page; README, "Test"): `pnpm lint`, `pnpm typecheck` and `pnpm build` pass and `pnpm test:unit` runs **91** tests. The mock-mode Playwright run passed **211 tests** (41 skipped), **221** (31 skipped) with `E2E_STRATEGY=1`; the live project, which starts its own API and web build, passed **96** (14 skipped), **104** (6 skipped) with the flag. Zero retries were the bar (handoff 09 section 3). They cover the demo walk with the golden numbers, the owner home, Deals, the documents page (read, review, confirm, reject, upload, in English and Chinese), Activity, the chat panel, owner sign-in, the approval queue, phone layouts, axe on every page and the adapter contract. The UX remediation's own gate (10 Oct) had run 6,398 API tests and a cold clone with every

page answering 200 (`docs/UX_REMEDIATION_REPORT.md` section 12). No screen-reader session, zoom or real-device pass was run and no WCAG conformance is claimed (`docs/SYSTEM_WALKTHROUGH.md` section 12). [TEAM: rerun them at the submitted commit.]

Scripted tasks and evidence. `scripts/run_scenarios.py --inproc` passes **49/49** tasks (87 `/nl` turns, 3 Oct 2026); all 49 evidence chains verify with `scripts/verify_evidence.py`. The nine tasks added on 1 Oct cover the financing comparison (en and zh), reconciliation, covenant ratios (en and zh), a capex case, the borrower file (now built at once, read-only), the nearest-capability fallback, and the agent key being told to ask the owner; three later tasks cover chat refusing to record a stated fact (en and zh) and reconciliation showing a bank reference. Details are in the Execution Evidence document.

Known limitations.

- **Free-text understanding is imperfect** (section 03). On the only untuned run, 52% verdict agreement and 4 wrong autonomous accepts out of 8. ADR-0005 now sends anything the compiler cannot fully account for to the owner, so the cost of a misread is a click, but the compiler still misses the material-error and silent-error targets on held-out 3.
- **Contract reading is shown, not measured.** The reading was recorded with the deterministic grammar (no model key), written against the three demo documents; the contracts are English; scanned PDFs are refused. No accuracy figure is quoted until contracts written by someone who has not seen the schema exist (ADR-0012, Consequences; `docs/handover/08_session_handoff_2026-10-12.md` section 2).
- **The strategy answers are flagged and unvalidated against real use.** First envelope read is 6 to 11 s cold on the golden books (`docs/STRATEGY_MVP_REPORT.md` section 8).
- **Time-bound offers cannot be accepted autonomously.** Settlement dates and windows are not read into terms, so any stated one fails the coverage proof and the offer waits for the owner.
- **Extra requests are detected by cues** (request verbs, questions and conditions, en + zh). A request with no cue and no figure could pass the proof; it is bounded to a pure early-pay discount below break-even.
- **Held-out 1 is no longer held out**; its near-perfect scores in `latest_report.md` are not accuracy claims. The report's findings table (EV-01 to EV-12) is the historical list from the first evaluation run; the summary table above it is the current run.
- **Open security risks** (Security Self-Assessment, section 04): single-operator key custody (R-2), the Tier-1 blast radius of a compromised agent (R-3), the trusted in-process actor (R-5), and the medium items found through chat (threshold probing S5 by a buyer with its key, the agent key reading status and alerts through chat S9). Channel authentication (R-4) and the plan binding of "go ahead" (R-12) were fixed on 3 Oct; the open operational rows (a multi-leg plan is not one transaction, a bank event without a reference is identified by its key only, unauthenticated intake capped per simulated day and bounded in size, and every caller rate-limited per minute, concurrency proven on SQLite and on a local PostgreSQL 17 with the PG-1 startup fix (merged), the live-model certification not run) are R-15. The hash chain is internally checkable only: its head is not anchored outside the system (audit S8), and head hashes differ from run to run. The PostgreSQL guards were verified on a local PostgreSQL 17 on 16 and 17 Oct but not in CI; since 17 Oct no session setting lifts them (F-11, SEC-08), and `/demo/reset` is owner-only and refused on a persistent store outside demo mode, but still erases the log in demo mode (F-05). The Security Lead's audit adds production blockers (no runnable production configuration, the chain

head not anchored, one process only) and certifies the local demo only (latest: [CERT: assessed build]).

- **Two accepted limits of the overhaul** (handoff 09 section 4). The outbound guard matches a protected contract date or "N months" as plain text, so a reply to another counterparty that happens to contain the same date is blocked: it fails closed (U-014). An offer that has arrived but has not been evaluated is known to the Deals read only from the session's documents, so it is not listed after a restart (U-015); in the Demo an offer is evaluated at once.
- **A benchmark that fails on one machine.** The EX01 evaluation benchmark (`tests/funding/test_benchmark.py`, 350 ms) ran at about 400 ms on the Windows machine before the overhaul and after it; `SKIP_PERF_BENCH=1` hides it (handoff 09 section 3).
- **Everything is simulated.** Lenders and the buyer are mocks, facilities and rates are synthetic, and there are no real bank integrations or loan applications.
- **Contract ingestion is shown, never booked.** A confirmed contract does not create a buyer's receivables or recurring invoices, and a lender agreement does not add or change a facility. Deliverables and payment documents are listed with their quotes, not tracked. Scanned PDFs are refused (no OCR). With no model key, the demo readings were recorded with the deterministic grammar, which was written against the three demo documents; the held-out contract set that would measure reading accuracy is pending, and its report quotes no figure (`services/api/evals/contracts_heldout_report.md`). The demo contracts are English; a Chinese contract has not been tried.
- **Covenant definitions differ by lender.** We test the defined covenants "under this definition", not bank-grade compliance; the system makes no claim of covenant compliance with any bank.
- **What the checks cover.** The projection screens the obligations that were entered. EBITDA is the owner's figure and recurring invoices are treated as confirmed sales collected on their due day, with one buyer as all of the receivables. "Cheapest" means cheapest within the bounded search.
- **Not available:** the after-signing extension (statutory due dates, late interest, receivable confirmation) is not built; its empty module and flag were removed. The mandate cannot be renewed by the agent: an alert warns 14 days before it expires and renewal is an offline owner step. The simulated clock, the bank feed and every receipt are simulated; no real payment is made or recorded.
- **Covenants on bullet loans.** A bullet term loan is serviced on a level basis for DSCR (ADR-0008), a reserve-coverage reading and not a lender's definition. The ¥500,000 12-month guaranteed loan therefore shows 1.014× and fails the 1.25× threshold. A bullet balance already drawn when a projection starts is treated as arriving today and spread over a full term, and a loan repaid early keeps its total interest spread over the whole term; both overstate service and are left as conservative.
- **Early-payment rule.** An invoice under an early-payment agreement is never financed, in the engine or the executor (conservative: it can refuse a cheap early payment a few days before the due day, never accept one wrongly). The alert cure search is greedy and can miss a cure that needs more than one invoice. An over-cap Tier-2 offer is queued as pending and is refused only when the owner approves it. Reconciliation reuses `SHORT_PAYMENT` for a partial payment.
- **The rate ceiling sends an expensive cure to the owner.** The day-34 advance of ¥200,000 costs ¥1,782.19, 32.5% a year for the 10 days it is held, above the mandate's 15% yearly ceiling because invoice financing is billed in whole 30-day periods. The opener says so; "go ahead"

queues the advance and it runs only when the owner approves it as a second step. The effective annual cost uses simple annualisation over the days held; a lender or regulator may define APR differently (U-009).

- **Annualised cost.** The API returns the annualised cost of each discount on owner verdicts (`annualised`), and the verdict page, each counter card and the chat show it next to the discount with the committed sources' yearly rates for comparison.
- **Live mode is opt-in and less tested.** The demo, the evidence and the evaluation tables use cache mode (recorded model outputs, an in-memory store); the only live accuracy run is the DeepSeek held-out-3 row. Live-model mode was not exercised for the 1 Oct and 3 Oct 2026 changes (the clock, the opener, the new chat intents, the consent grammar, the buyer channel key). The language-boundary certification harness is built and passes on recorded and scripted models; the live run is pending a key.
- **PostgreSQL is exercised locally, not in CI.** On 16 Oct the concurrency suite ran on a local PostgreSQL 17: the invariants hold once the PG-1 startup race is fixed (branch pending merge). A migrated Supabase database, connection pooling and several replicas were not tested, so the deployment should run one worker.